MACHINE LEARNING NOTES

Feature Engineering & Missing Data Handling

Videos 31–40 from

CampusX — 100 Days of Machine Learning

Chapter 31 — Power Transformer

Chapter 32 — Binning and Binarization

Chapter 33 — Handling Mixed Variables

Chapter 34 — Handling Date and Time Variables

Chapter 35 — Missing Data: Complete Case Analysis

Chapter 36 — Numerical Missing Data: SimpleImputer

Chapter 37 — Missing Categorical Data

Chapter 38 — Missing Indicator and Random Sample Imputation

Chapter 39 — KNN Imputer

Chapter 40 — MICE and Iterative Imputation

A complete, self-contained textbook following the teaching standards of the ML Roadmap project.

Table of Contents

Table of Contents 2

31.1 Introduction and Problem Statement 10

31.1.1 Why Skewness Matters for Machine Learning 10

31.2 Understanding Skewness 12

31.2.1 Mathematical Definition 12

31.2.2 Sample Skewness 12

31.2.3 Symmetric vs Asymmetric Distributions 12

31.3 Mathematical Foundations of Power Transformations 13

31.3.1 The Role of Lambda 13

31.3.2 Continuity at $\lambda = 0$ 14

31.4 Box-Cox Transformation 15

31.4.1 Definition 15

31.4.2 How Lambda is Estimated 15

31.4.3 Numerical Example 15

31.5 Yeo-Johnson Transformation 17

31.5.1 Definition 17

31.5.2 Box-Cox vs Yeo-Johnson — Direct Comparison 17

31.5.3 Lambda Interpretation for Yeo-Johnson 18

31.6 Scikit-learn PowerTransformer 19

31.6.1 Class Parameters 19

31.6.2 Key Attributes 19

31.6.3 Full Implementation Example 19

31.6.4 Box-Cox Implementation 20

31.6.5 Standardization Option 21

31.7 Before-and-After Comparison 22

31.7.1 Quantitative Assessment 22

31.8 Advantages, Limitations and Common Mistakes 23

31.8.1 Advantages 23

31.8.2 Limitations 23

31.8.3 Common Mistakes 23

31.8.4 Complete Pipeline Example 23

31.9 Key Takeaways 25

31.9.1 Quick Reference 25

31.9.2 Revision Questions 25

31.9.3 Interview Questions 25

32.1 Introduction: Continuous vs Discrete Variables 28

32.2 Why Bin Continuous Features? 28

32.3 Equal-Width Binning 29

32.4 Equal-Frequency (Quantile) Binning 29

32.4.1 Mathematical Definition 29

32.5 K-Means-Based Binning 31

32.6 Scikit-learn Implementation 31

32.6.1 KBinsDiscretizer 31

32.7 Binarization 32

32.7.1 Binarizer in Scikit-learn 32

32.8 Comparison of Techniques 33

32.9 Key Takeaways 33

32.10 Interview Questions 33

33.1 What is a Mixed Variable? 35

33.2 Why Mixed Variables Are Problematic 35

33.3 Feature Extraction and Splitting 36

33.3.1 Python Implementation 36

33.3.2 Regex Pattern Explanation 37

33.4 Handling Unit Conversions 38

33.5 Practical Considerations 38

33.6 Key Takeaways 38

34.1 Understanding DateTime Data 41

34.2 Converting Strings to DateTime Objects 41

34.3 Extracting Datetime Components 43

34.4 Cyclical Encoding of DateTime Features 45

34.5 Time-Based Feature Engineering 45

34.6 Key Takeaways 46

35.1 Understanding Missing Data 48

35.1.1 Why Do Missing Values Occur? 48

35.2 Missingness Mechanisms 49

35.3 Complete Case Analysis (CCA) 50

35.3.1 Python Implementation 50

35.4 When CCA is Appropriate **52**

35.5 When CCA Should be Avoided **52**

35.6 Key Takeaways **52**

35.7 Interview Questions **53**

36.1 Introduction to Imputation **55**

36.2 Mean vs Median Imputation **55**

36.2.1 Mathematical Basis 55

36.3 Scikit-learn SimpleImputer **56**

36.3.1 Full Implementation 56

36.4 Column-Wise Imputation **58**

36.5 Statistical Implications **58**

36.6 Key Takeaways **58**

36.7 Interview Questions **59**

37.1 Missing Categorical Values **61**

37.2 Strategies for Missing Categorical Data **61**

37.3 Implementation **62**

37.3.1 Most Frequent Imputation 62

37.3.2 Constant / Missing Category Imputation 62

37.4 Consequences of Different Approaches **64**

37.4.1 Most Frequent Imputation 64

37.4.2 Constant / Missing Category 64

37.5 Key Takeaways 64

38.1 Missing Indicators 66

38.2 MissingIndicator in Scikit-learn 66

38.3 Random Sample Imputation 68

38.3.1 Implementation 68

38.4 Advantages and Limitations 70

38.4.1 Missing Indicator 70

38.4.2 Random Sample Imputation 70

38.5 Key Takeaways 70

39.1 Understanding K-Nearest Neighbors 72

39.2 Mathematical Foundation: Distance 73

39.2.1 Distance with Missing Values 73

39.3 KNNImputer in Scikit-learn 74

39.4 Weights Parameter 76

39.4.1 Full Pipeline Example 76

39.5 Advantages, Disadvantages and Computational Cost 78

39.5.1 Advantages 78

39.5.2 Disadvantages 78

39.6 Key Takeaways 78

39.7 Interview Questions 78

40.1 Why Univariate Imputation May Be Insufficient 80

40.2 MICE: Multivariate Imputation by Chained Equations 80

40.3 The MICE Algorithm 81

40.3.1 Mathematical Intuition 81

40.4 IterativeImputer in Scikit-learn 82

40.5 Comparison: KNN vs MICE 84

40.6 Convergence and Stopping Criteria 85

40.7 Advantages and Assumptions 85

40.7.1 Advantages 85

40.7.2 Assumptions 85

40.8 Key Takeaways 85

40.9 Interview Questions 85

R.1 Comparison of All Missing-Data Methods 87

R.1.1 Decision Framework 88

R.2 Key Formulas Reference 89

R.2.1 Skewness 89

R.2.2 Box-Cox Transformation 89

R.2.3 Yeo-Johnson Transformation ($x \geq 0$) 89

R.2.4 Euclidean Distance 89

R.2.5 Cyclical Encoding 89

R.2.6 KNN Weighted Imputation 89

R.3 Scikit-learn Classes Reference 90

R.4 Quick Interview Cheat Sheet 90

CHAPTER 31

Power Transformer

Skewness, Box-Cox & Yeo-Johnson Transformations

Learning Objectives
Why skewed distributions hurt ML models; how power transformations restore symmetry; the mathematics of Box-Cox and Yeo-Johnson; Scikit-learn implementation; when to choose one method over the other.

31.1 Introduction and Problem Statement

Most machine-learning algorithms make assumptions about the shape of the data they receive. Linear regression, for example, assumes that the relationship between features and the target is linear and that residuals are normally distributed. Logistic regression and many distance-based methods implicitly benefit from features that are roughly symmetric. When a feature is heavily skewed — meaning most values cluster on one side with a long tail on the other — these assumptions break down, and model performance can degrade significantly.

Skewness is a measure of the asymmetry of a probability distribution around its mean. A perfectly symmetric distribution (like the Gaussian / normal distribution) has a skewness of zero. A distribution with a long right tail has positive skewness; one with a long left tail has negative skewness.

In real-world datasets, skewed features are extremely common. Income data is almost always right-skewed (a few people earn very large amounts). House prices are right-skewed. Transaction amounts, response times, and many sensor readings also exhibit skewness. If we feed such features directly into a model, a small number of extreme values can dominate the learning process, pull the decision boundary, and reduce overall accuracy.

Power transformations are a family of monotonic mathematical functions that map a continuous variable to a new scale in a way that makes its distribution more symmetric (closer to a normal distribution). The transformation applies the same function to every value in the feature, preserving the rank order of the observations while reshaping the overall distribution.

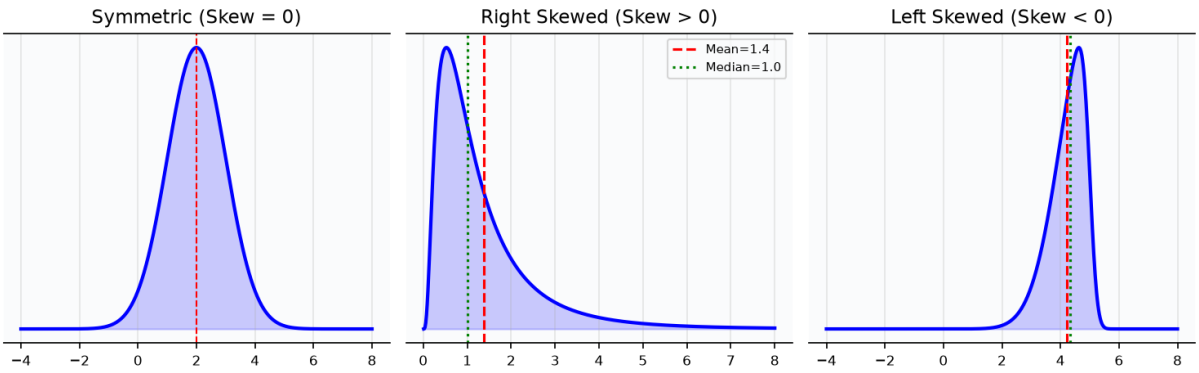


Figure 31.1: Three fundamental distribution shapes. Left: symmetric (mean = median, skewness = 0). Centre: right-skewed / positive skew (mean > median, long right tail). Right: left-skewed / negative skew (mean < median, long left tail).

31.1.1 Why Skewness Matters for Machine Learning

Consider a linear-regression model that predicts house price from square footage. If square footage is heavily right-skewed, most houses cluster around a small value while a few mansions stretch far to the right. The ordinary least-squares criterion minimises the sum of squared residuals, so extreme points contribute quadratically more to the loss. The fitted line is therefore pulled toward those extremes, and predictions for typical houses become less accurate.

Distance-based algorithms such as k-nearest neighbours, k-means clustering, and support vector machines are even more sensitive. They compute Euclidean distances between feature values. A single feature with a range of 0–1,000,000 will dwarf a feature with a range of 0–1, effectively silencing the smaller feature entirely.

Tree-based methods (decision trees, random forests, gradient boosting) are less affected because they split on rank order rather than absolute values. However, even tree-based models can benefit from power transformations when the transformed feature yields more informative split points.

Key Insight

Power transformations do not change the **rank order** of observations. If observation A had a larger value than observation B before the transformation, it will still have a larger value after. This monotonicity property is what makes power transformations safe for most machine-learning pipelines.

31.2 Understanding Skewness

31.2.1 Mathematical Definition

The **skewness** of a random variable X with mean μ and standard deviation σ is defined as the normalised third central moment:

$$\text{Skewness}(X) = \frac{E\left[(X - \mu)^3\right]}{\sigma^3} \tag{31.1}$$

where E[·] denotes the expected value (population mean). The numerator is the third central moment, and dividing by σ^3 makes the measure dimensionless so it can be compared across features with different units.

Interpretation

- **Skewness ≈ 0**: The distribution is approximately symmetric.
- **Skewness > 0** (positive): The right tail is longer; the mass of the distribution is concentrated on the left. Mean > Median.
- **Skewness < 0** (negative): The left tail is longer; the mass is on the right. Mean < Median.

In practice, a common rule of thumb is that absolute skewness less than 0.5 indicates an approximately symmetric distribution, values between 0.5 and 1 indicate moderate skewness, and values above 1 indicate high skewness that warrants transformation.

31.2.2 Sample Skewness

For a sample of n observations, the sample skewness (Fisher-Pearson coefficient) is computed as:

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left[\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right]^{3/2}} \tag{31.2}$$

Pandas provides this directly via `df['col'].skew()`. Scipy provides it via `scipy.stats.skew(data)`.

31.2.3 Symmetric vs Asymmetric Distributions

A distribution is **symmetric** if it looks the same on the left and right sides of its centre. The normal (Gaussian) distribution is the classic example. For a symmetric distribution, the mean, median, and mode all coincide.

An **asymmetric** (skewed) distribution has different behaviour on each side. In a right-skewed distribution the mean is pulled toward the long right tail, so mean > median. In a left-skewed distribution the mean is pulled toward the long left tail, so mean < median. This relationship between mean and median is a quick diagnostic for skewness direction.

31.3 Mathematical Foundations of Power Transformations

A **power transformation** is a function of the form:

$$T(x;\lambda) = \begin{cases} \dfrac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x) & \text{if } \lambda = 0 \end{cases} \tag{31.3}$$

where x is the original value and λ (lambda) is a real-valued parameter that controls the shape of the transformation. Different values of λ produce different transformations from the same family.

31.3.1 The Role of Lambda

Lambda is the free parameter that the algorithm estimates from the data. The goal is to choose λ so that the transformed data is as close to normally distributed as possible. Each classical choice of λ has an intuitive meaning:

Lambda (λ)	Transformation	Formula	Use Case
λ = 2	Square	x^2	Left-skewed data (mild)
λ = 1	Identity (no change)	x	Data already symmetric
λ = 0.5	Square root	√x	Moderately right-skewed count data
λ = 0	Natural logarithm	ln(x)	Heavily right-skewed data
λ = –1	Reciprocal	1/x	Strongly right-skewed data

Table 31.1: Common lambda values and their corresponding transformations.

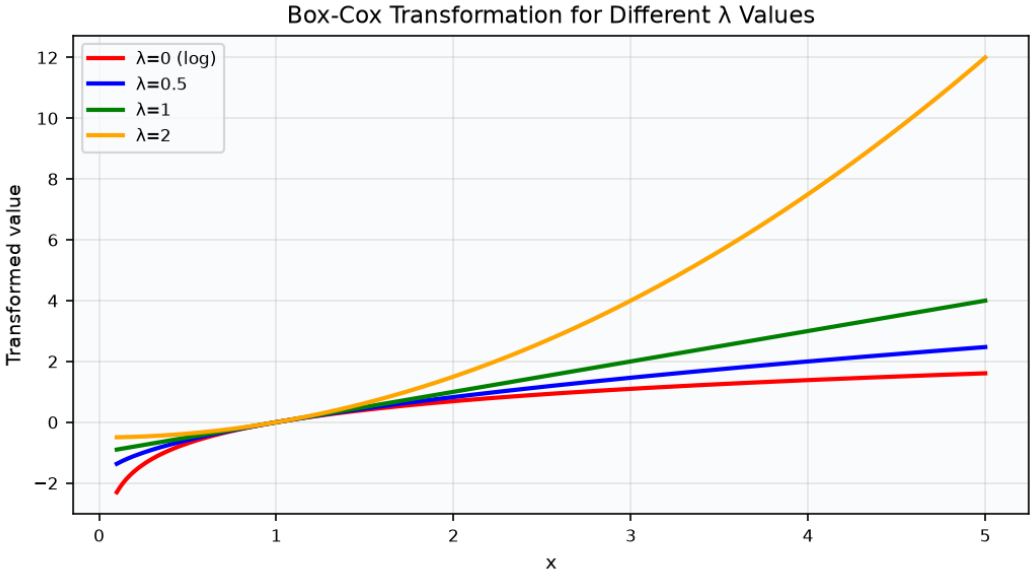


Figure 31.2: The Box-Cox transformation applied to x ∈ (0, 5) for different λ values. As λ decreases from 2 to 0, the curve becomes increasingly compressive on the right tail — exactly what is needed to tame right-skewed data.

The function T(x; λ) is **monotonic increasing** for all λ. This means it preserves the ordering of data points: if x■ < x■ then T(x■; λ) < T(x■; λ). This property is crucial because it means no information about relative ranking is lost during the transformation.

31.3.2 Continuity at $\lambda = 0$

You might notice that the formula divides by λ, which would be undefined at $\lambda = 0$. However, taking the limit as $\lambda \to 0$ using L'Hôpital's rule gives:

$$\lim_{\lambda \to 0} \frac{x^{\lambda} - 1}{\lambda} = \lim_{\lambda \to 0} \frac{\frac{d}{d\lambda}(x^{\lambda} - 1)}{\frac{d}{d\lambda}(\lambda)} = \lim_{\lambda \to 0} \frac{x^{\lambda} \ln x}{1} = \ln x \tag{31.4}$$

This is why the log transformation ($\lambda = 0$) is defined as a separate case — it is the continuous limit of the power transformation as λ approaches zero.

31.4 Box-Cox Transformation

The Box-Cox transformation, introduced by George Box and David Cox in 1964, is the most widely used power transformation. It finds the optimal λ by maximising the log-likelihood of the transformed data under a normal model.

31.4.1 Definition

For a vector of positive observations x = (x■, x■, …, x■), all x■ > 0, the Box-Cox transformation of each element is:

$$y_i = T(x_i; \lambda) = \begin{cases} \dfrac{x_i^\lambda - 1}{\lambda} & \lambda \neq 0 \\[6pt] \ln(x_i) & \lambda = 0 \end{cases} \tag{31.5}$$

> **Critical Restriction**
> The Box-Cox transformation is **only defined for strictly positive values** (x■ > 0 for all i). If any value in the feature is zero or negative, Box-Cox cannot be applied directly. A common workaround is to shift all values by a constant: x' = x – min(x) + δ where δ is a small positive number. However, this shifts the optimal λ and can affect results. The Yeo-Johnson transformation (Section 31.5) solves this problem entirely.

31.4.2 How Lambda is Estimated

Box and Cox proposed estimating λ by maximising the log-likelihood function under the assumption that the transformed data follow a normal distribution. The log-likelihood is:

$$\ell(\lambda) = -\frac{n}{2}\ln\left(\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2\right) + (\lambda - 1)\sum_{i=1}^{n}\ln(x_i) \tag{31.6}$$

where y■ = T(x■; λ) and ■ is the mean of the transformed values. The first term captures how well the transformed data fits a normal distribution (smaller residual variance is better). The second term is a Jacobian correction that accounts for the change of variable — it prevents the maximisation from simply shrinking all values toward zero.

In practice, the optimisation is performed numerically over a grid of λ values (typically from –5 to +5), or using Newton-Raphson / gradient-based optimisation. Scipy's `stats.boxcox()` and Scikit-learn's `PowerTransformer` both use efficient numerical methods internally.

31.4.3 Numerical Example

Suppose we have five positive observations and want to find the optimal λ by hand (simplified grid search):

```
import numpy as np
from scipy.stats import boxcox

# Five positive observations (right-skewed)
data = np.array([1.2, 3.5, 7.8, 15.2, 45.6, 8.1, 12.3, 2.4, 20.5, 5.7])

# Find optimal lambda and transformed data
transformed, optimal_lambda = boxcox(data)

print(f"Optimal lambda: {optimal_lambda:.4f}")
print(f"Original data: {data}")
print(f"Transformed: {transformed}")
```

Listing 31.1: Estimating the optimal lambda with scipy.stats.boxcox.

The function returns the transformed array and the estimated λ. You can then apply the same λ to new data using the inverse operation, or simply refit the transformer on the training set and apply it to test data.

31.5 Yeo-Johnson Transformation

The Yeo-Johnson transformation, introduced by Yeo and Johnson in 2000, is a generalisation of the Box-Cox transformation that works for **all real numbers**, including zero and negative values. This is its primary advantage over Box-Cox.

31.5.1 Definition

For any real value x and parameter λ, the Yeo-Johnson transformation is defined piecewise:

(31.7)

Notice how the formula switches its form depending on whether x is non-negative or negative. For $x \geq 0$ it behaves like a shifted Box-Cox on $(x + 1)$. For $x < 0$ it uses a different power that involves $(2 - \lambda)$. This two-piece construction ensures the transformation is defined, continuous, and monotonically increasing for all real x.

31.5.2 Box-Cox vs Yeo-Johnson — Direct Comparison

Property	Box-Cox	Yeo-Johnson
Input domain	x > 0 only	All real numbers (x ∈ ■)
Handles zeros?	No (must shift)	Yes
Handles negatives?	No	Yes
Formula complexity	Simpler (single expression)	More complex (4-piece)
Estimation method	Maximise log-likelihood	Maximise log-likelihood
Scikit-learn method	method="box-cox"	method="yeo-johnson"
Default in sklearn?	No	Yes
Introduced	1964	2000

Table 31.2: Comparison of Box-Cox and Yeo-Johnson transformations.

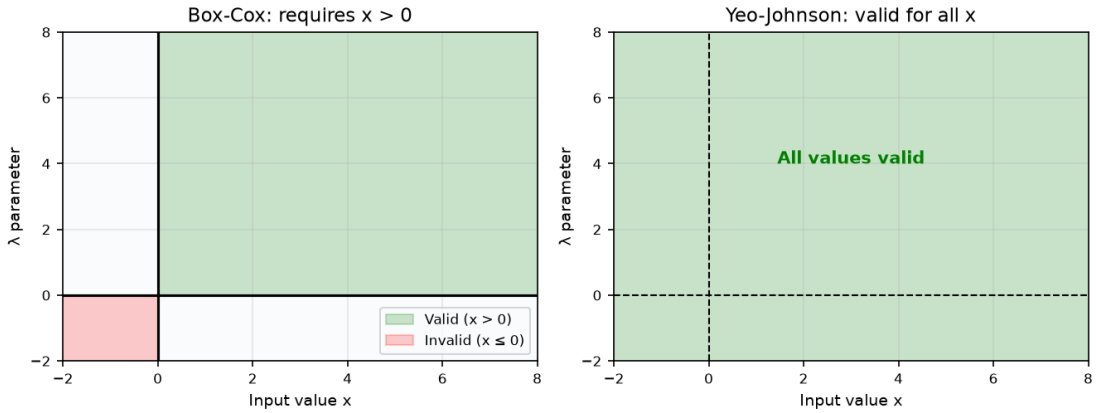


Figure 31.3: Domain of validity. Box-Cox (left) requires all input values to be strictly positive (x > 0). Yeo-Johnson (right) is valid for all real numbers including zero and negatives.

31.5.3 Lambda Interpretation for Yeo-Johnson

The interpretation of λ is similar to Box-Cox but not identical. For Yeo-Johnson:

- $\lambda = 1 \rightarrow$ identity (no transformation)
- $\lambda = 0 \rightarrow$ approximately log-like transformation for positive values
- $\lambda = 0.5 \rightarrow$ approximately square-root transformation for positive values
- $\lambda = 2 \rightarrow$ approximately square transformation for positive values

The exact mapping differs slightly from Box-Cox because of the $(x + 1)$ shift in the formula. The estimated λ should always be checked against the transformed distribution rather than interpreted in isolation.

31.6 Scikit-learn PowerTransformer

Scikit-learn provides the `PowerTransformer` class in `sklearn.preprocessing`. It supports both Box-Cox and Yeo-Johnson methods and automatically estimates the optimal λ during the `fit()` step.

31.6.1 Class Parameters

Parameter	Default	Description
method	"yeo-johnson"	Transformation type: "box-cox" or "yeo-johnson"
standardize	True	If True, center to mean=0 and scale to variance=1 after transform

Table 31.3: PowerTransformer parameters.

31.6.2 Key Attributes

Attribute	Description
lambda_	Estimated λ for each feature (array of shape [n_features,])
n_features_in_	Number of features seen during fit
feature_names_in_	Feature names (if fitted with named features)

Table 31.4: PowerTransformer attributes after fitting.

31.6.3 Full Implementation Example

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PowerTransformer
from scipy.stats import skew

# Step 1: Create a realistically skewed dataset
np.random.seed(42)
n_samples = 1000

# Income data — typically right-skewed (log-normal)
income = np.random.lognormal(mean=10.5, sigma=0.8, size=n_samples)

# Add a second feature with moderate skew
age_squared = np.random.uniform(18, 70, n_samples) ** 1.5 / 100

df = pd.DataFrame({
'income': income,
'age_transformed': age_squared
})

print("=== BEFORE TRANSFORMATION ===")
print(f"Skewness (income): {df['income'].skew():.4f}")
print(f"Skewness (age): {df['age_transformed'].skew():.4f}")
print(f"Income range: {df['income'].min():.0f} to {df['income'].max():.0f}")
print()

# Step 2: Apply Yeo-Johnson transformation
```

```
pt = PowerTransformer(method='yeo-johnson')
df_transformed = pd.DataFrame(
pt.fit_transform(df),
columns=df.columns
)

print("=== AFTER YEO-JOHNSON TRANSFORMATION ===")
print(f"Skewness (income): {df_transformed['income'].skew():.4f}")
print(f"Skewness (age): {df_transformed['age_transformed'].skew():.4f}")
print(f"Estimated lambdas: {pt.lambda_}")
print()

# Step 3: Visualize before and after
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Before
axes[0, 0].hist(df['income'], bins=50, color='steelblue', edgecolor='white')
axes[0, 0].set_title(f'Income BEFORE\nSkewness = {df["income"].skew():.2f}')
axes[0, 0].set_xlabel('Income ($)')

axes[0, 1].hist(df['age_transformed'], bins=50, color='coral', edgecolor='white')
axes[0, 1].set_title(f'Age Feature BEFORE\nSkewness = {df["age_transformed"].skew():.2f}')

# After
axes[1, 0].hist(df_transformed['income'], bins=50, color='steelblue', edgecolor='white')
axes[1, 0].set_title(f'Income AFTER Yeo-Johnson\nSkewness =
{df_transformed["income"].skew():.2f}')
axes[1, 0].set_xlabel('Transformed Value')

axes[1, 1].hist(df_transformed['age_transformed'], bins=50, color='coral',
edgecolor='white')
axes[1, 1].set_title(f'Age Feature AFTER Yeo-Johnson\nSkewness =
{df_transformed["age_transformed"].skew():.2f}')

plt.tight_layout()
plt.savefig('power_transform_result.png', dpi=150)
plt.show()
```

Listing 31.2: Complete PowerTransformer workflow with visualization.

31.6.4 Box-Cox Implementation

```
from sklearn.preprocessing import PowerTransformer

# Box-Cox requires ALL values to be strictly positive
pt_bc = PowerTransformer(method='box-cox')

# This will raise an error if any value <= 0:
# pt_bc.fit_transform(df_with_negative_values) # ValueError!

# For positive-only data:
pt_bc.fit(df_positive)
print(f"Box-Cox lambdas: {pt_bc.lambda_}")

# Transform the data
df_bc = pd.DataFrame(
pt_bc.transform(df_positive),
columns=df_positive.columns
)
```

Listing 31.3: Box-Cox implementation with positivity check.

31.6.5 Standardization Option

When `standardize=True` (the default), the transformer additionally scales the output so that each feature has mean = 0 and variance = 1. This is a two-step process:

1. Apply the power transformation T(x; λ) to each value.
2. Standardise the result: z = (y – ■) / s, where ■ is the sample mean and s is the sample standard deviation of the transformed values.

If you only want the power transformation without standardisation, set `standardize=False`. This is useful when you plan to apply a separate scaling method (e.g., MinMaxScaler) later in the pipeline.

31.7 Before-and-After Comparison

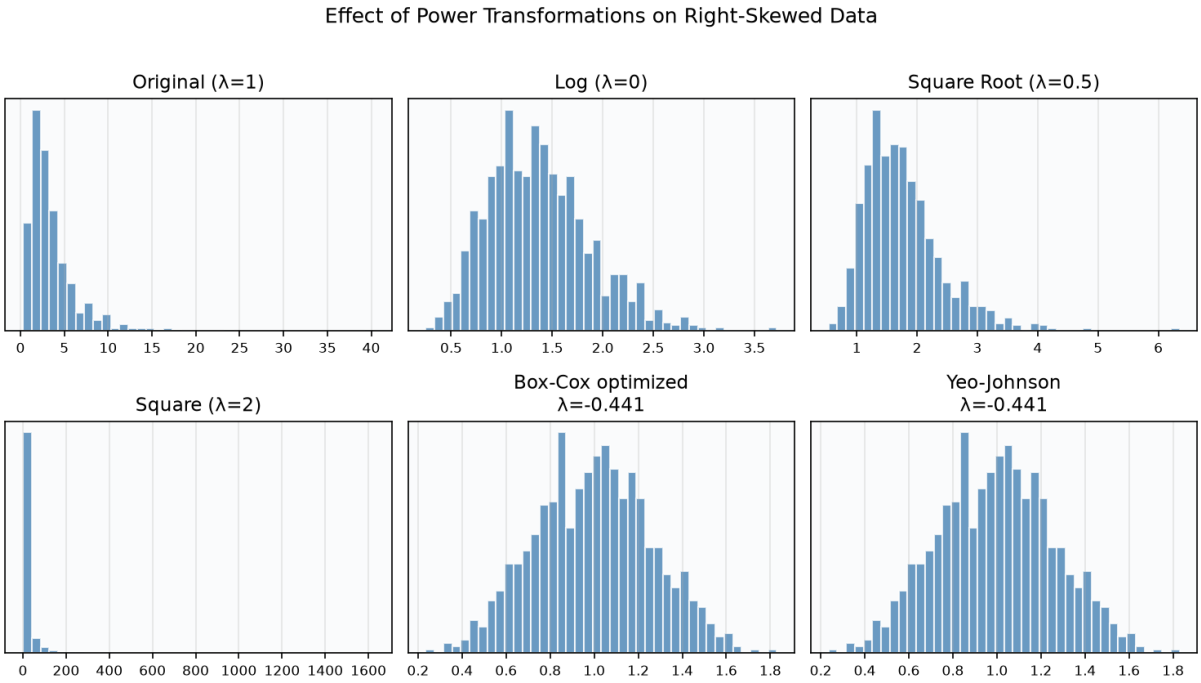


Figure 31.4: Six panels showing the effect of different power transformations on the same right-skewed dataset. The top-left panel shows the original distribution. Each other panel shows the result of applying a different λ. The Box-Cox and Yeo-Johnson panels use the automatically estimated optimal λ.

31.7.1 Quantitative Assessment

After transformation, you should verify the improvement quantitatively:

```
from scipy.stats import skew, kurtosis, shapiro

def assess_normality(data, name="Feature"):
"""Print skewness, kurtosis, and Shapiro-Wilk test."""
s = skew(data)
k = kurtosis(data) # excess kurtosis (0 for normal)
stat, p_value = shapiro(data[:500]) # Shapiro limited to 500 samples
print(f"{name}:")
print(f" Skewness: {s:.4f} (ideal: 0)")
print(f" Kurtosis: {k:.4f} (ideal: 0)")
print(f" Shapiro-Wilk p-value: {p_value:.4f} (>0.05 suggests normal)")
print()

# Assess before
assess_normality(df['income'], "Income BEFORE")

# Assess after
assess_normality(df_transformed['income'], "Income AFTER PowerTransformer")
```

Listing 31.4: Quantitative assessment of normality before and after transformation.

After a successful power transformation, you should observe that the absolute skewness has decreased substantially (ideally toward zero), the kurtosis has moved closer to zero (the excess kurtosis of a normal distribution), and the Shapiro-Wilk test p-value has increased (though with n=1000 the test is very sensitive and may still reject perfect normality).

31.8 Advantages, Limitations and Common Mistakes

31.8.1 Advantages

- **Model-agnostic:** Power transformations can be applied before any machine-learning algorithm, not just specific ones.
- **Monotonic:** Rank order is preserved, so no information about relative ordering is lost.
- **Automatic λ estimation:** Scikit-learn automatically finds the optimal λ during fitting — no manual tuning required.
- **Interpretable:** The estimated λ tells you how much skewness correction was needed.
- **Computationally cheap:** The transformation is a simple element-wise operation — O(n) time and space.

31.8.2 Limitations

- **Box-Cox requires positive data:** If your feature contains zeros or negatives, use Yeo-Johnson instead.
- **Does not guarantee normality:** Power transformations make the distribution more symmetric but cannot always achieve perfect normality, especially for multimodal distributions.
- **Data leakage risk:** The λ is estimated from the data. You must fit the transformer on the training set only and apply it to the test set. Fitting on the entire dataset leaks information.
- **Interpretability loss:** After transformation, feature values are on a new scale. Coefficients of linear models refer to the transformed scale, which is harder to explain to stakeholders.
- **Not useful for tree-based models:** Decision trees, random forests, and gradient boosting are invariant to monotonic transformations, so power transformations provide no benefit for these models.

31.8.3 Common Mistakes

Mistake 1: Fitting on the entire dataset
Always split your data into training and test sets FIRST, then fit the PowerTransformer on the training set only. If you fit on the entire dataset, the estimated λ and the standardisation statistics (mean, std) leak information from the test set into the training process.

Mistake 2: Using Box-Cox on non-positive data
Box-Cox requires all values to be strictly positive. If your data contains zeros or negatives, Scikit-learn will raise a ValueError. Either switch to Yeo-Johnson or carefully shift the data (knowing this affects the result).

Mistake 3: Forgetting to inverse-transform predictions
If you transform the target variable, you must inverse-transform the model predictions before evaluation. Otherwise, your metrics (MSE, MAE) will be on the transformed scale, not the original scale.

31.8.4 Complete Pipeline Example

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Synthetic dataset with skewed features
np.random.seed(42)
n = 1000
X = pd.DataFrame({
'income': np.random.lognormal(10.5, 0.8, n),
'spending': np.random.lognormal(8.0, 1.0, n),
'age': np.random.uniform(18, 70, n)
})
y = (2.5 * np.log(X['income']) + 1.5 * np.log(X['spending']) +
0.1 * X['age'] + np.random.normal(0, 1, n))

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

# Fit transformer on TRAINING data only
pt = PowerTransformer(method='yeo-johnson')
X_train_transformed = pt.fit_transform(X_train)
X_test_transformed = pt.transform(X_test) # Apply, do not fit!

print(f"Estimated lambdas: {pt.lambda_}")

# Train model on transformed features
model = LinearRegression()
model.fit(X_train_transformed, y_train)

# Predict and evaluate
y_pred = model.predict(X_test_transformed)
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
```

Listing 31.5: Proper train/test pipeline with PowerTransformer.

31.9 Key Takeaways

> **★ Power transformations reshape skewed distributions toward symmetry by applying a monotonic function controlled by a single parameter λ. Box-Cox works for positive data only; Yeo-Johnson works for all real numbers. Always fit on training data and apply to test data.**

31.9.1 Quick Reference

Concept	Key Point
Skewness	Measure of distribution asymmetry; 0 = symmetric
Box-Cox	Requires x > 0; optimises log-likelihood over λ
Yeo-Johnson	Works for all real x; piecewise formula
Scikit-learn	PowerTransformer(method="yeo-johnson"|"box-cox")
Lambda attribute	pt.lambda_ after fitting
Standardize	Default True: center to 0, scale to variance 1
Data leakage	Fit on train, transform on test — never fit on test
Tree models	No benefit — trees are invariant to monotonic transforms

Table 31.5: Quick reference for Power Transformer concepts.

31.9.2 Revision Questions

1. What is skewness and how do you measure it?
2. Why do power transformations help linear regression but not decision trees?
3. State the Box-Cox formula and explain the role of λ.
4. What is the main restriction of Box-Cox, and how does Yeo-Johnson overcome it?
5. What does PowerTransformer(standardize=True) do after the power transformation?
6. Why must you fit the transformer only on training data?
7. If the estimated λ for a feature is 0, what transformation was effectively applied?
8. How would you handle a feature that contains both positive and negative values when you want to apply a power transformation?

31.9.3 Interview Questions

1. **Q:** What is the difference between Box-Cox and Yeo-Johnson?
A: Box-Cox is defined only for positive values and uses a single formula; Yeo-Johnson is defined for all real numbers using a piecewise formula that switches form for $x \geq 0$ and $x < 0$.
2. **Q:** How does Scikit-learn estimate λ in PowerTransformer?
A: By maximising the log-likelihood of the transformed data under a normal model, using numerical optimisation.

3. **Q:** Can you apply PowerTransformer to the target variable in regression?
A: Yes, but you must inverse-transform predictions before computing metrics on the original scale.

4. **Q:** When would you *not* use a power transformation?
A: With tree-based models (no benefit), with categorical features (not continuous), or when interpretability of original-scale coefficients is required.

CHAPTER 32

Binning and Binarization

Discretization, Equal-Width, Quantile, K-Means Binning

Learning Objectives
Continuous vs discrete variables; equal-width, equal-frequency, and K-Means binning; binarization; Scikit-learn KBinsDiscretizer and Binarizer; comparison of techniques.

32.1 Introduction: Continuous vs Discrete Variables

Machine-learning algorithms consume numerical features. However, not every continuous feature benefits from being continuous. Sometimes we deliberately convert a continuous variable into a discrete one — a process called **discretization** — to reduce noise, handle non-linear relationships, or make the feature more interpretable.

A **continuous variable** can take any value within a range (e.g., age = 25.7, temperature = 36.6). A **discrete variable** takes only a finite set of values (e.g., age group = "young", "middle", "senior"). **Binning** is the process of converting the former into the latter by partitioning the value range into intervals.

Binarization is the extreme case of binning where the variable is reduced to exactly two categories: 0 or 1, based on a threshold.

32.2 Why Bin Continuous Features?

- **Reduce noise:** Small fluctuations within a bin are absorbed, making the feature more robust to measurement error.
- **Handle non-linearity:** If the relationship between feature and target is stepwise rather than linear, binning captures the steps directly.
- **Improve interpretability:** "Age group: senior" is easier to communicate than "Age: 67.3".
- **Manage outliers:** Extreme values are compressed into the edge bins, reducing their influence.
- **Satisfy algorithm assumptions:** Some algorithms (e.g., naive Bayes with Gaussian assumption) work better with binned features.

> **Caution: Information Loss**
> Binning is a form of information loss. The continuous value within each bin is discarded. If the continuous feature is highly informative, binning can reduce model performance. Always validate with cross-validation whether binning improves your specific model.

32.3 Equal-Width Binning

Equal-width binning divides the range of the feature into k intervals of equal width. If the feature ranges from min to max, the bin edges are:

$$\text{width} = \frac{\max(x) - \min(x)}{k}, \quad \text{edges} = \left[\min, \min + w, \min + 2w, \ldots, \max\right] \tag{32.1}$$

For example, with values ranging from 0 to 100 and k = 4 bins, the edges are [0, 25, 50, 75, 100], giving intervals [0–25], [25–50], [50–75], [75–100].

Equal-width binning works well when the data is uniformly distributed. However, if the data is skewed, some bins may contain many observations while others are nearly empty — a problem that equal-frequency binning solves.

32.4 Equal-Frequency (Quantile) Binning

Equal-frequency binning (also called quantile binning) divides the data so that each bin contains approximately the same number of observations. The bin edges are the quantiles of the feature distribution.

For k bins, the edges are at the quantiles 0, 1/k, 2/k, …, 1. For 4 bins, the edges are the 0th (min), 25th, 50th (median), 75th, and 100th (max) percentiles.

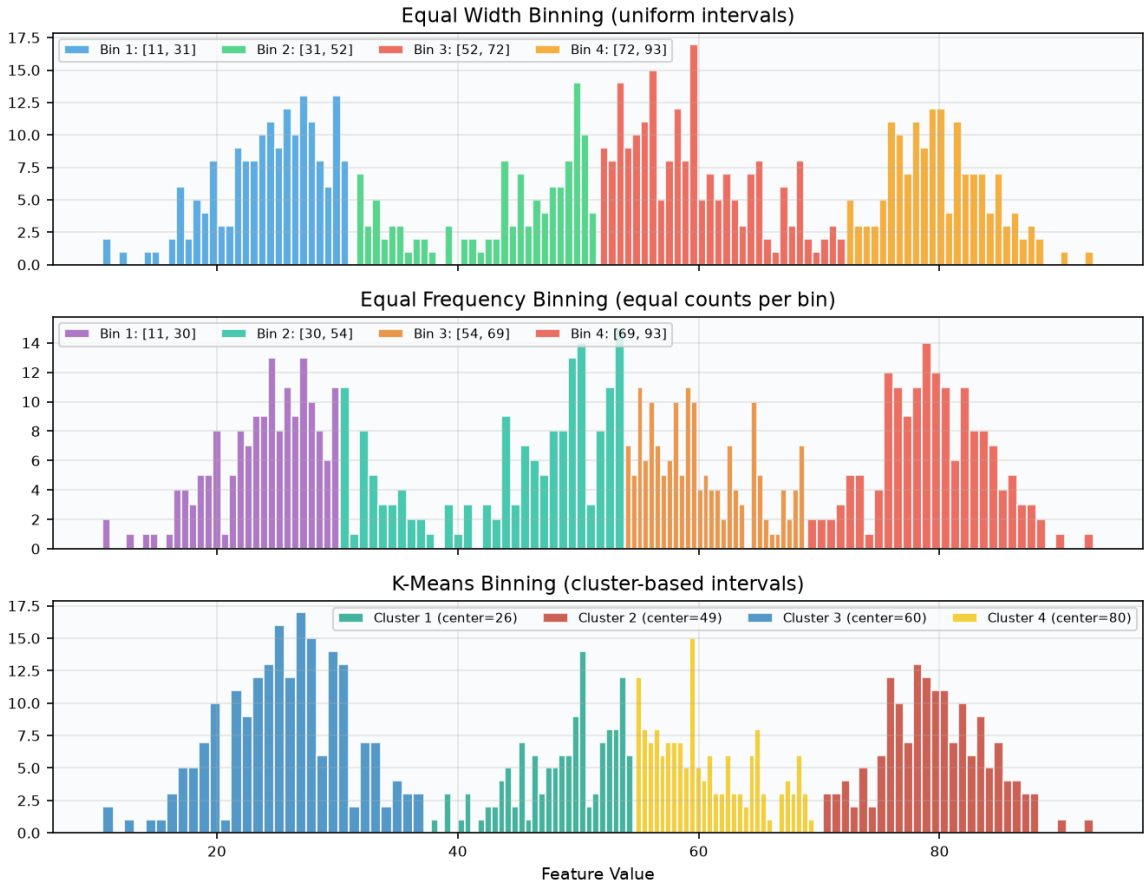


Figure 32.1: Three binning strategies applied to the same multimodal distribution. Top: equal-width (uniform intervals, unequal counts). Middle: equal-frequency (equal counts, unequal intervals). Bottom: K-Means-based (cluster centres determine intervals).

32.4.1 Mathematical Definition

For k bins, the j-th bin edge (j = 0, 1, …, k) is the quantile at level j/k:

$$e_j = Q\left(\frac{j}{k}\right), \quad j = 0, 1, \ldots, k \tag{32.2}$$

where Q(p) is the p-th quantile of the feature distribution. An observation x■ belongs to bin j if e■ ≤ x■ < e■■■ (with the last bin including the maximum).

32.5 K-Means-Based Binning

A third strategy uses **K-Means clustering** to determine the bin edges. Instead of using fixed-width or quantile-based intervals, K-Means finds k cluster centres that minimise the within-cluster sum of squared distances. The bin edges are then placed at the midpoints between adjacent cluster centres.

This approach has an advantage: it adapts to the actual density of the data. Dense regions get narrower bins (more resolution), while sparse regions get wider bins. The result is often more informative than equal-width or equal-frequency binning.

32.6 Scikit-learn Implementation

32.6.1 KBinsDiscretizer

Parameter	Default	Description
n_bins	5	Number of bins per feature
encode	"ordinal"	Encoding: "ordinal" (0,1,2...), "onehot", "onehot-dense"
strategy	"uniform"	Binning strategy: "uniform", "quantile", "kmeans"

Table 32.1: KBinsDiscretizer parameters.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer
import matplotlib.pyplot as plt

# Create skewed data
np.random.seed(42)
data = np.random.lognormal(mean=2, sigma=1, size=1000).reshape(-1, 1)

# Apply three binning strategies
strategies = ['uniform', 'quantile', 'kmeans']
fig, axes = plt.subplots(3, 2, figsize=(10, 10))

for idx, strategy in enumerate(strategies):
disc = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy=strategy)
binned = disc.fit_transform(data)

# Plot original
axes[idx, 0].hist(data, bins=50, color='steelblue', edgecolor='white')
axes[idx, 0].set_title(f'{strategy.title()} - Original')

# Plot binned distribution
unique, counts = np.unique(binned, return_counts=True)
axes[idx, 1].bar(unique, counts, color='coral', edgecolor='white')
axes[idx, 1].set_title(f'{strategy.title()} - After Binning')
axes[idx, 1].set_xlabel('Bin Index')
axes[idx, 1].set_ylabel('Count')

# Print bin edges
print(f"{strategy.title()} bin edges: {disc.bin_edges_[0]}")

plt.tight_layout()
```

```
plt.savefig('binning_strategies.png', dpi=150)
plt.show()
```

Listing 32.1: Comparing all three KBinsDiscretizer strategies.

32.7 Binarization

Binarization converts a continuous feature into a binary feature (0 or 1) using a threshold. Values above the threshold become 1; values at or below become 0.

$$x_{\text{binary}} = \begin{cases} 1 & \text{if } x > \text{threshold} \\ 0 & \text{if } x \leq \text{threshold} \end{cases} \tag{32.3}$$

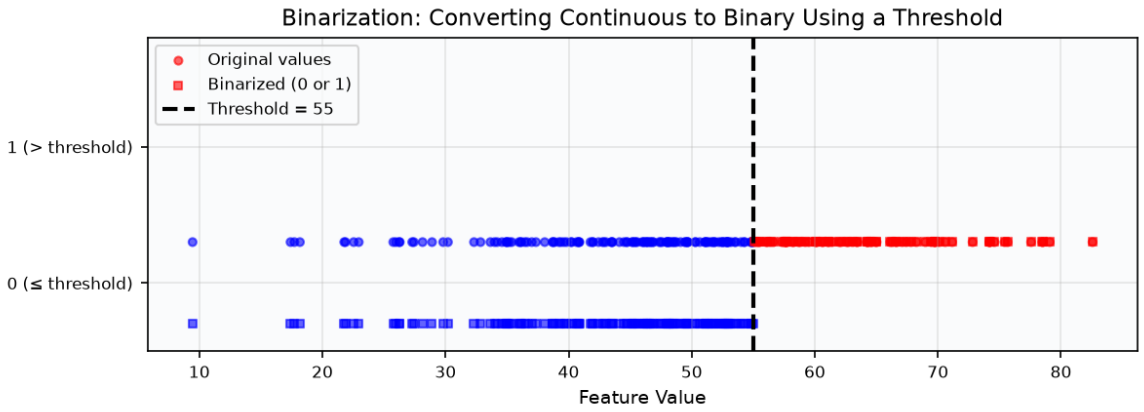


Figure 32.2: Binarization with threshold = 55. Original continuous values (top row) are mapped to 0 or 1 (bottom row) based on whether they exceed the threshold.

32.7.1 Binarizer in Scikit-learn

```
from sklearn.preprocessing import Binarizer
import numpy as np

data = np.array([[10], [25], [35], [50], [65], [80], [95]])

# Threshold at 50
binarizer = Binarizer(threshold=50)
binary = binarizer.fit_transform(data)

print("Original:", data.flatten())
print("Binary: ", binary.flatten())
# Output: Original: [10 25 35 50 65 80 95]
# Binary: [ 0 0 0 0 1 1 1]
```

Listing 32.2: Binarizer with threshold=50.

> **Important: Binarizer Threshold**
> The Binarizer uses a **strictly greater-than** comparison: values equal to the threshold are assigned 0. If you want values ≥ threshold to be 1, set threshold to (desired_threshold - small_epsilon).

32.8 Comparison of Techniques

Method	How edges are determined	Best for	Drawback
Equal-width	Uniform intervals across [min, max]	Uniformly distributed data	Skewed data → uneven bin counts
Equal-frequency	Quantiles (equal count per bin)	Skewed data, robust to outliers	May split identical values across bins
K-Means	Cluster centres → midpoints as edges	Multimodal / naturally clustered data	Computationally heavier; requires k
Binarization	Single threshold → 0 or 1	Yes/no decisions, simple rules	Extreme information loss

Table 32.2: Comparison of binning and binarization techniques.

32.9 Key Takeaways

> **★ Binning converts continuous features into discrete bins to reduce noise and handle non-linearity. Equal-width uses uniform intervals, equal-frequency uses quantiles, and K-Means adapts to data density. Binarization is the extreme case with a single threshold producing 0/1 output.**

32.10 Interview Questions

1. What is the difference between equal-width and equal-frequency binning?
2. When would you prefer K-Means binning over quantile binning?
3. Does binning always improve model performance? Explain.
4. How does Binarizer handle values exactly equal to the threshold?
5. What does KBinsDiscretizer(encode="onehot") produce?

CHAPTER 33

Handling Mixed Variables

Feature Extraction, Splitting, Unit Conversion, Encoding

Learning Objectives
What mixed variables are; why they are problematic; feature splitting with regex; unit normalisation; encoding strategies after splitting.

33.1 What is a Mixed Variable?

A **mixed variable** (also called a mixed-type feature) is a single feature that contains both numerical and categorical information simultaneously. Examples include "5 feet 7 inches" (number + unit), "2024-01-15" (numbers + separators), or "40.7°N" (number + direction).

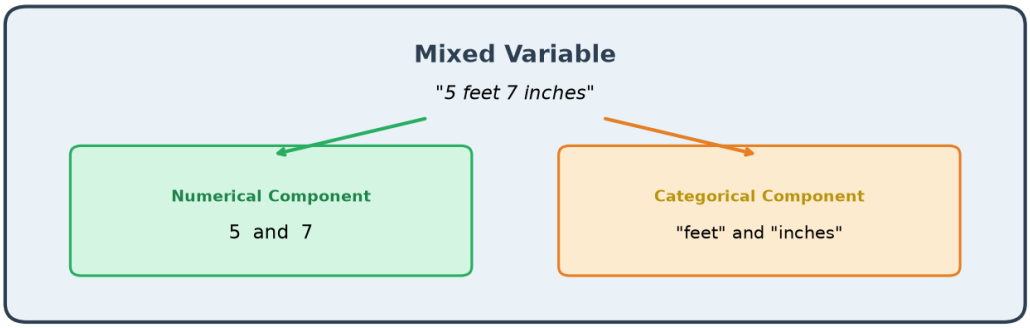


Figure 33.1: Anatomy of a mixed variable. "5 feet 7 inches" contains numerical components (5, 7) and categorical components (feet, inches) within a single feature column.

Common Examples of Mixed Variables

Height	`5 feet 7 inches`	*Numeric + Unit category*
Weight	`70.5 kg`	*Numeric + Unit category*
Date-Time	`2024-01-15 14:30:00`	*Numeric parts + separators*
Coordinates	`40.7128° N, 74.0060° W`	*Numeric + Direction category*
File Size	`2.5 MB`	*Numeric + Unit category*
Range	`10-20`	*Two numerics + separator*

Figure 33.2: Common examples of mixed variables found in real datasets.

33.2 Why Mixed Variables Are Problematic

Machine-learning algorithms expect each feature to have a single data type. A mixed variable cannot be directly interpreted as numeric (because it contains text) or as categorical (because it contains meaningful numbers). Feeding a mixed variable as-is into a model will either cause an error or force the model to treat every unique string as a separate category — leading to high cardinality and poor generalization.

33.3 Feature Extraction and Splitting

The primary strategy for handling mixed variables is **feature splitting**: extract the numerical parts into one feature and the categorical parts into another. This way each resulting feature has a single, clear type.

Before Splitting

Height_Mixed
5 feet 7 inches
6 feet 2 inches
5 feet 1 inch

After Feature Splitting

Height_Value	Height_Unit
5, 7	feet, inches
6, 2	feet, inches
5, 1	feet, inches

Figure 33.3: Before splitting, a single mixed column contains both numbers and text. After splitting, two clean columns result: one numeric (Height_Value) and one categorical (Height_Unit).

33.3.1 Python Implementation

```
import pandas as pd
import numpy as np
import re

# Sample mixed dataset
data = {
'height': ['5 feet 7 inches', '6 feet 2 inches', '5 feet 1 inch',
'5 feet 9 inches', '6 feet 0 inches', '5 feet 5 inches'],
'weight': ['70.5 kg', '82.3 kg', '55.0 kg', '91.2 kg', '68.7 kg', '73.1 kg'],
'temperature': ['36.6 C', '37.2 C', '38.1 C', '35.9 C', '36.0 C', '37.8 F']
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print(f"\nDtypes:\n{df.dtypes}")

# ■■ Split height into value and unit ■■
df[['height_value', 'height_unit']] = df['height'].str.extract(
r'(\d+(?:\.\d+)?)\s*(feet|inches?)', expand=True
)

# Convert height_value to numeric
df['height_value'] = pd.to_numeric(df['height_value'])

# ■■ Split weight into value and unit ■■
df[['weight_value', 'weight_unit']] = df['weight'].str.extract(
r'([\d.]+)\s*(kg|lbs?)', expand=True
)
df['weight_value'] = pd.to_numeric(df['weight_value'])

# ■■ Split temperature ■■
df[['temp_value', 'temp_unit']] = df['temperature'].str.extract(
r'([\d.]+)\s*([CF])', expand=True
)
df['temp_value'] = pd.to_numeric(df['temp_value'])
```

```
print("\nAfter Feature Splitting:")
print(df[['height_value', 'height_unit',
'weight_value', 'weight_unit',
'temp_value', 'temp_unit']])
```

Listing 33.1: Splitting mixed variables using regex extraction.

33.3.2 Regex Pattern Explanation

The regular expression `r'(\d+(?:\.\d+)?)\s*(feet|inches?)'` works as follows:

- `\d+` — matches one or more digits
- `(?:\.\d+)?` — optionally matches a decimal point followed by digits (for values like 5.5)
- `\s*` — matches zero or more whitespace characters
- `(feet|inches?)` — captures either "feet" or "inch"/"inches"

33.4 Handling Unit Conversions

After splitting, the unit column may contain different units for the same feature (e.g., "feet" vs "inches", "kg" vs "lbs", "C" vs "F"). You should normalise these to a single unit before using the numerical values.

```
# Convert all heights to centimeters
def height_to_cm(row):
value = row['height_value']
unit = row['height_unit']
if unit == 'feet':
return value * 30.48 # 1 foot = 30.48 cm
elif unit in ('inch', 'inches'):
return value * 2.54 # 1 inch = 2.54 cm
return np.nan

df['height_cm'] = df.apply(height_to_cm, axis=1)

# Convert all temperatures to Celsius
def temp_to_celsius(row):
value = row['temp_value']
unit = row['temp_unit']
if unit == 'C':
return value
elif unit == 'F':
return (value - 32) * 5 / 9
return np.nan

df['temp_celsius'] = df.apply(temp_to_celsius, axis=1)

print(df[['height_value', 'height_unit', 'height_cm']])
print()
print(df[['temp_value', 'temp_unit', 'temp_celsius']])
```

Listing 33.2: Normalising units after feature splitting.

33.5 Practical Considerations

- **Always split before encoding:** Attempting to one-hot encode a mixed variable directly will treat each unique string as a category.
- **Validate extracted values:** Regex extraction can fail on unexpected formats. Check for NaN values after extraction.
- **Consider the domain:** Sometimes the categorical part carries more information than the numerical part (e.g., unit type might correlate with target).
- **Preserve the original:** Keep the original mixed column until you have verified the split features are correct.
- **Automated approaches:** For large-scale data, consider libraries like `feature-engine` or custom transformers that can be integrated into a sklearn Pipeline.

33.6 Key Takeaways

> **★ Mixed variables combine numerical and categorical information in one feature. The primary solution is feature splitting: extract numeric parts into one column and categorical parts into another using regex or string methods, then normalise units and encode categories separately.**

CHAPTER 34

Handling Date and Time Variables

DateTime Parsing, Feature Extraction, Cyclical Encoding

Learning Objectives
Converting strings to datetime; Pandas datetime accessor; extracting components; cyclical encoding with sin/cos; time-based feature engineering.

34.1 Understanding DateTime Data

Date and time columns are ubiquitous in real-world datasets: transaction timestamps, user activity logs, sensor readings, weather records, and more. While they may appear as simple strings, datetime values encode a wealth of information that can be extracted into predictive features.

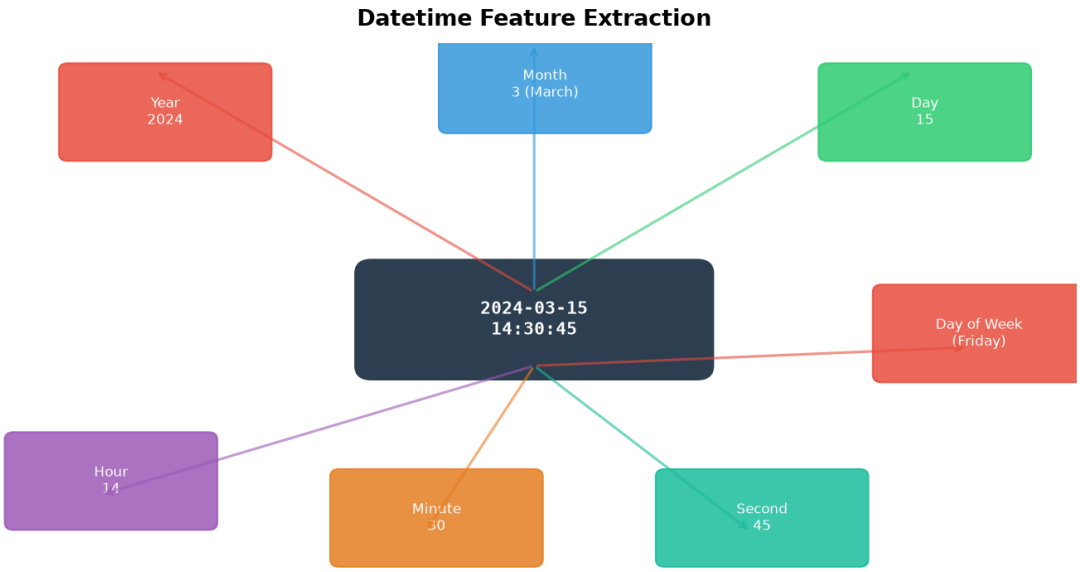


Figure 34.1: A single datetime value can be decomposed into year, month, day, hour, minute, second, and day-of-week — each a potential feature for machine learning.

34.2 Converting Strings to DateTime Objects

```
import pandas as pd
import numpy as np

# Various datetime string formats
dates = pd.Series([
'2024-01-15',
'15/01/2024',
'Jan 15, 2024',
'2024-01-15 14:30:00',
'2024-01-15T14:30:00.000Z',
'15-01-2024 14:30'
])

# Convert to datetime objects
dt = pd.to_datetime(dates, format='mixed', dayfirst=True)
print("Converted datetime objects:")
print(dt)
print(f"\nDtype: {dt.dtype}")

# Access datetime components
print(f"\nYear: {dt.dt.year.tolist()}")
print(f"Month: {dt.dt.month.tolist()}")
print(f"Day: {dt.dt.day.tolist()}")
print(f"Hour: {dt.dt.hour.tolist()}")
```

Listing 34.1: Converting various string formats to datetime objects.

Common DateTime Error

If pandas cannot infer the date format, it raises a `ParserError`. Always specify `format=` when the format is known, or use `format='mixed'` (pandas ≥ 2.0) for mixed-format columns. Also watch for day-first vs month-first ambiguity: "01/02/2024" could be 1 Feb or 2 Jan depending on locale.

34.3 Extracting Datetime Components

Attribute / Method	Returns	Example (2024-03-15 14:30:45)
.dt.year	Year as integer	2024
.dt.month	Month (1-12)	3
.dt.day	Day of month (1-31)	15
.dt.hour	Hour (0-23)	14
.dt.minute	Minute (0-59)	30
.dt.second	Second (0-59)	45
.dt.dayofweek	Day of week (Mon=0, Sun=6)	4 (Friday)
.dt.day_name()	Day name string	"Friday"
.dt.dayofyear	Day of year (1-366)	75
.dt.is_month_start	First day of month?	False
.dt.is_month_end	Last day of month?	False
.dt.is_weekend	Saturday or Sunday?	False
.dt.quarter	Quarter (1-4)	1
.dt.days_in_month	Days in the month	31
.dt.to_period("M")	Period (month)	2024-03

Table 34.1: Useful Pandas datetime accessor attributes.

```
import pandas as pd

# Sample transaction timestamps
df = pd.DataFrame({
'transaction_id': range(1, 11),
'timestamp': pd.to_datetime([
'2024-01-15 09:30:00', '2024-01-16 14:22:00',
'2024-01-17 18:45:00', '2024-01-18 11:15:00',
'2024-01-19 20:30:00', '2024-01-20 08:00:00',
'2024-01-21 16:10:00', '2024-01-22 13:55:00',
'2024-01-23 19:20:00', '2024-01-24 10:40:00'
]),
'amount': [120.50, 89.99, 250.00, 45.00, 310.75,
199.99, 75.25, 420.00, 160.00, 95.50]
})

# Extract features
df['year'] = df['timestamp'].dt.year
df['month'] = df['timestamp'].dt.month
df['day'] = df['timestamp'].dt.day
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['day_name'] = df['timestamp'].dt.day_name()
df['is_weekend'] = df['timestamp'].dt.dayofweek >= 5
df['quarter'] = df['timestamp'].dt.quarter
```

```
# Time-based bins
df['time_of_day'] = pd.cut(df['hour'],
bins=[0, 6, 12, 18, 24],
labels=['Night', 'Morning', 'Afternoon', 'Evening'])

print(df[['timestamp', 'hour', 'day_name', 'is_weekend', 'time_of_day', 'amount']])
print(f"\nAverage amount by time of day:")
print(df.groupby('time_of_day')['amount'].agg(['mean', 'count']))
```

Listing 34.2: Extracting datetime features for ML.

34.4 Cyclical Encoding of DateTime Features

Features like month (1–12) and hour (0–23) are **cyclical**: December (12) is close to January (1), and hour 23 is close to hour 0. If you encode these as plain integers, the model sees December and January as far apart — which is wrong.

The solution is **cyclical encoding** using sine and cosine transforms:

$$x_{\sin} = \sin\left(\frac{2\pi \cdot x}{\text{period}}\right), \quad x_{\cos} = \cos\left(\frac{2\pi \cdot x}{\text{period}}\right) \tag{34.1}$$

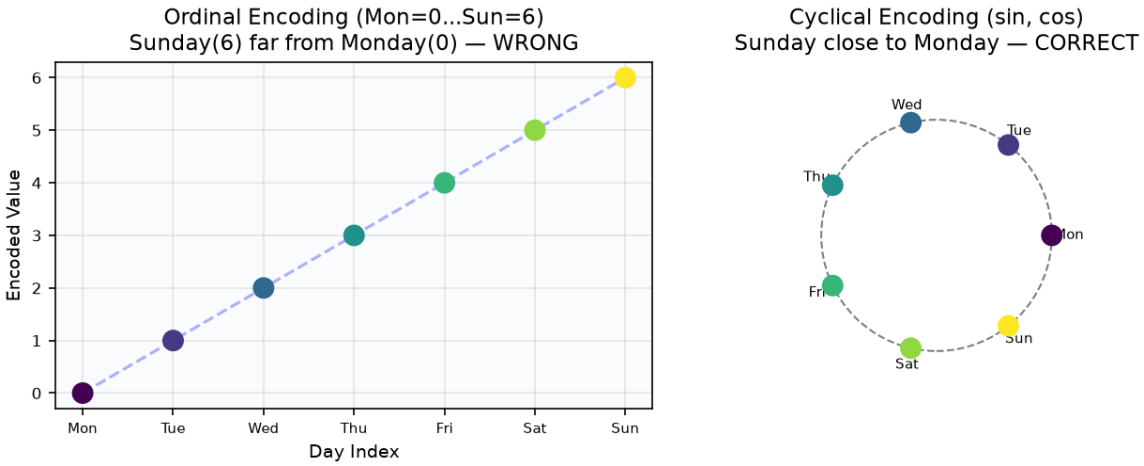


Figure 34.2: Left: ordinal encoding places Sunday (6) far from Monday (0), even though they are adjacent days. Right: cyclical (sin/cos) encoding maps days onto a circle where Sunday and Monday are neighbours.

```
# Cyclical encoding for hour (period=24) and month (period=12)
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)

# Verify: hours 23 and 0 should be close
h23 = df[df['hour'] == 23][['hour_sin', 'hour_cos']].values
h0 = np.array([np.sin(0), np.cos(0)])
print(f"Hour 23 encoding: {h23}")
print(f"Hour 0 encoding: {h0}")
print(f"Distance: {np.linalg.norm(h23 - h0):.4f} (small = close)")
```

Listing 34.3: Cyclical encoding with sine and cosine.

34.5 Time-Based Feature Engineering

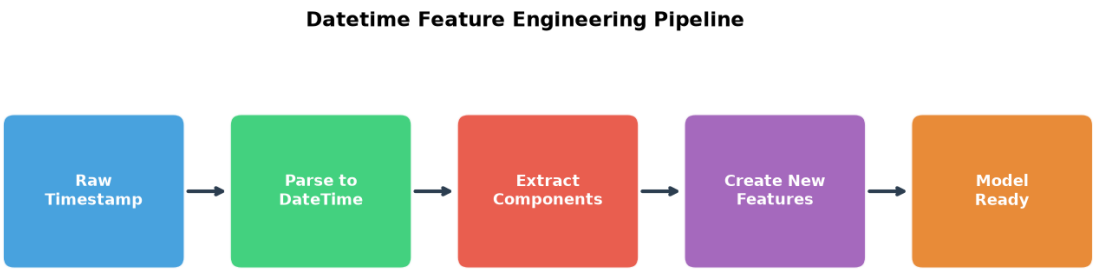


Figure 34.3: The datetime feature engineering pipeline from raw timestamp to model-ready features.

- **Lag features:** Value at time t–1, t–2, … (for time-series forecasting)
- **Rolling statistics:** Moving average over a window (e.g., 7-day mean)
- **Elapsed time:** Days since a reference event (e.g., account creation)
- **Recency:** Hours since last transaction
- **Frequency:** Transactions per day/week/month
- **Holidays and events:** Boolean flags for public holidays

34.6 Key Takeaways

> **★ DateTime columns should be parsed with pd.to_datetime, decomposed into meaningful components (year, month, day, hour, day-of-week), and cyclical features encoded with sine/cosine. Time-based features like lags, rolling means, and recency add predictive power for sequential data.**

CHAPTER 35

Missing Data: Complete Case Analysis

Missingness Mechanisms, MCAR/MAR/MNAR, CCA

Learning Objectives
Why missing values occur; MCAR, MAR, MNAR mechanisms; when CCA is valid; when to avoid CCA; decision framework for handling strategies.

35.1 Understanding Missing Data

Missing data is one of the most common problems in real-world machine learning. A value is "missing" when no information is recorded for a particular observation on a particular feature. In Pandas, missing values are represented as `NaN` (Not a Number) for numeric data and `None` for object data. In Scikit-learn, missing values are represented as `np.nan`.

35.1.1 Why Do Missing Values Occur?

- **Sensor failure:** A physical sensor malfunctioned during recording.
- **Human error:** A data-entry operator skipped a field.
- **Survey non-response:** A respondent left a question blank.
- **Data integration:** Merging datasets from different sources where some fields do not overlap.
- **Privacy restrictions:** Sensitive fields were deliberately withheld.
- **Conditional collection:** A follow-up question was only asked if the previous answer was "yes".

35.2 Missingness Mechanisms

Understanding *why* data is missing is crucial because it determines which handling methods are statistically valid. Rubin (1976) classified missingness into three mechanisms:

Mechanism	Abbreviation	Definition	Example
Missing Completely at Random	MCAR	Missingness is independent of both observed and unobserved data	Random equipment malfunction
Missing at Random	MAR	Missingness depends on observed data but not on the missing value itself	Younger patients skip income question more often
Missing Not at Random	MNAR	Missingness depends on the missing value itself	High-income earners refuse to report income

Table 35.1: Rubin's three missingness mechanisms.

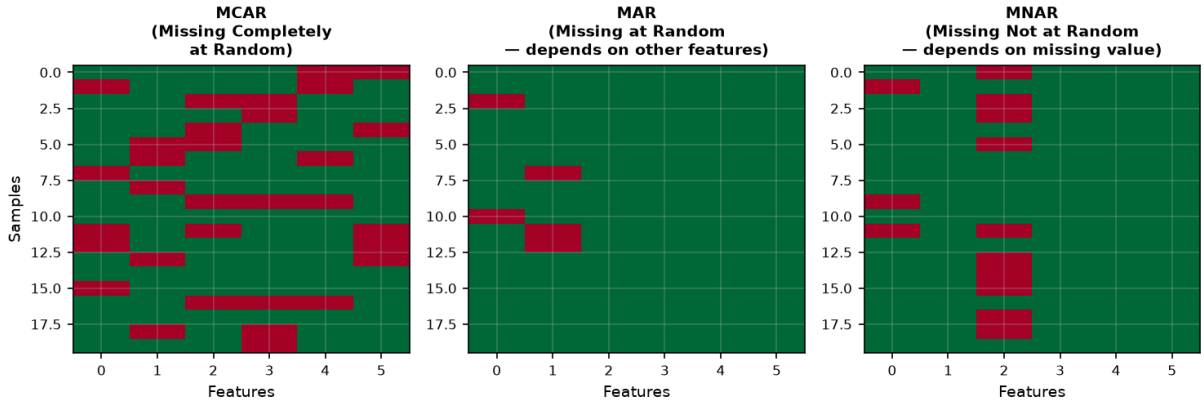


Figure 35.1: Visual representation of missing data patterns. Green cells are observed; red cells are missing. MCAR shows random scatter. MAR shows dependence on other observed features. MNAR shows dependence on the unobserved value itself (often harder to detect).

Why This Matters
If data is **MCAR**, removing incomplete rows (CCA) produces unbiased results. If data is **MAR** or **MNAR**, CCA can introduce bias because the missing pattern is informative. Understanding the mechanism guides the choice of method.

35.3 Complete Case Analysis (CCA)

Complete Case Analysis (also called listwise deletion) removes any row that contains at least one missing value. Only rows where every feature is observed are retained.

$$\text{CCA keeps row } i \iff \forall j:\ x_{ij} \neq \text{NaN} \tag{35.1}$$

35.3.1 Python Implementation

```
import pandas as pd
import numpy as np

# Create dataset with missing values
np.random.seed(42)
n = 200
df = pd.DataFrame({
'age': np.random.normal(35, 10, n),
'income': np.random.lognormal(10.5, 0.7, n),
'score': np.random.normal(70, 15, n),
'city': np.random.choice(['NYC', 'LA', 'Chicago', 'Houston'], n)
})

# Introduce missing values (MCAR - random)
for col in ['age', 'income', 'score']:
mask = np.random.random(n) < 0.15
df.loc[mask, col] = np.nan

print("=== MISSING DATA SUMMARY ===")
print(df.isnull().sum())
print(f"\nTotal rows: {len(df)}")
print(f"Rows with any missing: {df.isnull().any(axis=1).sum()}")
print(f"Percentage missing rows: {df.isnull().any(axis=1).mean()*100:.1f}%")

# Apply CCA
df_complete = df.dropna()
print(f"\n=== AFTER CCA ===")
print(f"Rows remaining: {len(df_complete)}")
print(f"Rows removed: {len(df) - len(df_complete)}")
print(f"Data loss: {(1 - len(df_complete)/len(df))*100:.1f}%")

# Compare statistics before and after
print(f"\n=== STATISTICAL COMPARISON ===")
print(f"{'Feature':<10} {'Full Mean':>10} {'CCA Mean':>10} {'Full Std':>10} {'CCA Std':>10}")
for col in ['age', 'income', 'score']:
print(f"{col:<10} {df[col].mean():>10.2f} {df_complete[col].mean():>10.2f} "
f"{df[col].std():>10.2f} {df_complete[col].std():>10.2f}")
```

Listing 35.1: Complete Case Analysis implementation.

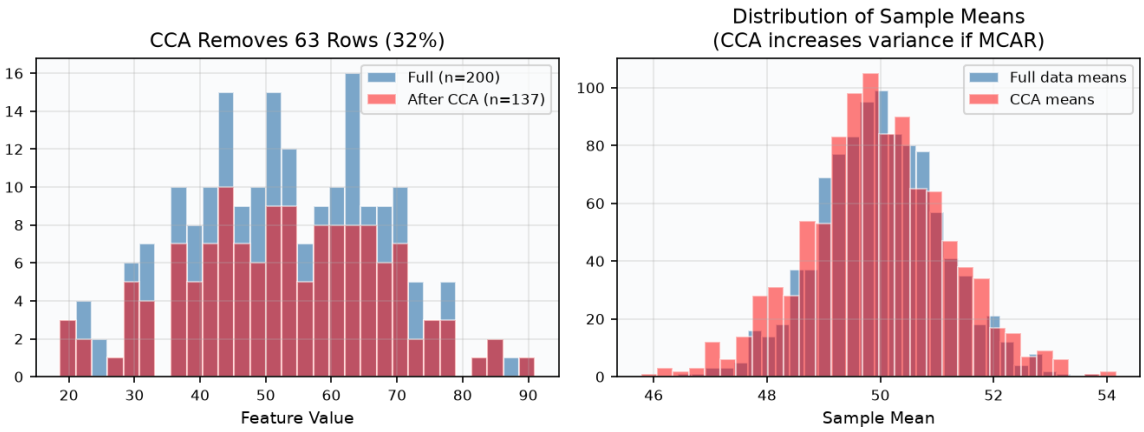


Figure 35.2: Left: CCA removes a substantial fraction of data. Right: Under MCAR, CCA preserves the distribution shape but reduces sample size, increasing the variance of parameter estimates.

35.4 When CCA is Appropriate

CCA is statistically valid when:

- The missingness mechanism is **MCAR** (missing completely at random).
- The proportion of missing data is **small** (typically < 5%).
- The dataset is **large enough** that losing some rows does not critically reduce statistical power.
- The missing values are not systematically related to the target variable.

35.5 When CCA Should be Avoided

- Missingness is **MAR** or **MNAR** — deletion introduces bias.
- The **percentage of missing data is high** (e.g., > 25%), leading to substantial information loss.
- The dataset is already **small** — every row matters.
- Missingness is **concentrated in important features** — dropping those rows removes valuable observations.
- The missing pattern is **systematic** — e.g., all patients who answered "yes" to a screening question have a follow-up value missing.

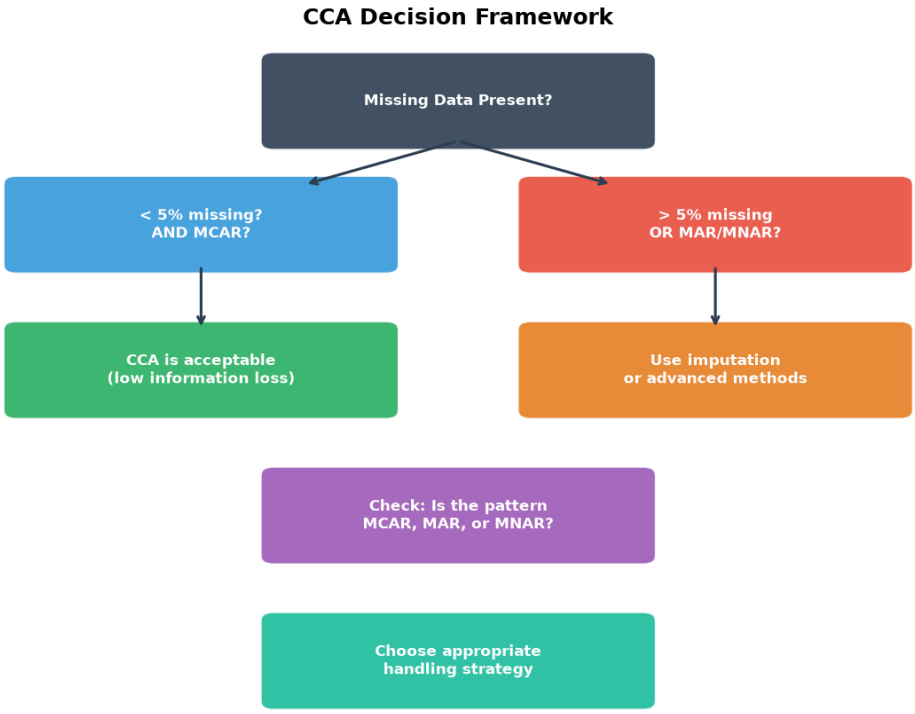


Figure 35.3: Decision framework for choosing between CCA and imputation.

35.6 Key Takeaways

★ Complete Case Analysis removes all rows containing any missing value. It is valid only when data is MCAR and the missing proportion is small. For MAR/MNAR or high missingness, imputation methods (Chapters 36–40) are preferred.

35.7 Interview Questions

1. Explain the difference between MCAR, MAR, and MNAR with examples.
2. Under what conditions is Complete Case Analysis statistically valid?
3. What happens to the variance of parameter estimates when you apply CCA?
4. How would you test whether data is MCAR?
5. Why is CCA biased when data is MNAR?

CHAPTER 36

Numerical Missing Data: SimpleImputer

Mean Imputation, Median Imputation, Strategy, Statistics

Learning Objectives
Mean vs median imputation; when median is preferred; SimpleImputer parameters and attributes; column-wise imputation; statistical implications.

36.1 Introduction to Imputation

When Complete Case Analysis is not appropriate — because too much data would be lost, or because the missingness mechanism is MAR — we turn to **imputation**: replacing missing values with estimated ones. The simplest and most widely used imputation method is **mean imputation** (replace NaN with the column mean) or **median imputation** (replace NaN with the column median).

Scikit-learn provides the `SimpleImputer` class for this purpose. Despite its name, SimpleImputer is a foundational tool that every ML practitioner must understand deeply.

36.2 Mean vs Median Imputation

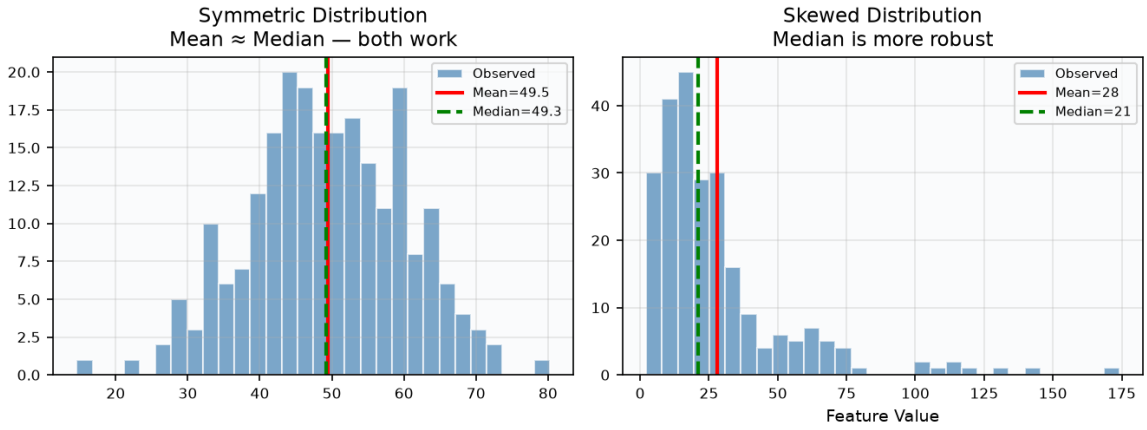


Figure 36.1: Left: for symmetric distributions, mean and median are nearly identical — either works. Right: for skewed distributions, the mean is pulled toward the long tail while the median remains robust. Median imputation is preferred for skewed data.

36.2.1 Mathematical Basis

Mean imputation replaces each missing value x■ with the sample mean of observed values in that feature:

$$\hat{x}_j = \bar{x}_{\text{obs}} = \frac{1}{n_{\text{obs}}} \sum_{i \in \text{obs}} x_{ij} \tag{36.1}$$

Median imputation replaces each missing value with the median of observed values:

$$\hat{x}_j = \text{median}\left(\{x_{ij} : i \in \text{obs}\}\right) \tag{36.2}$$

The mean is sensitive to outliers because it uses all values equally in its computation. The median — the middle value when data is sorted — is robust to outliers because it depends only on the rank order, not the magnitudes.

36.3 Scikit-learn SimpleImputer

Parameter	Default	Description
missing_values	np.nan	The placeholder for missing values
strategy	"mean"	Imputation strategy: "mean", "median", "most_frequent", "constant"
fill_value	None	Value to use when strategy="constant"
copy	True	If True, creates a copy; if False, imputes in place
add_indicator	False	If True, adds binary missingness indicator columns

Table 36.1: SimpleImputer parameters.

Attribute	Description
statistics_	Imputation value for each feature (array of shape [n_features])
n_features_in_	Number of features seen during fit
indicator_	MissingIndicator instance (if add_indicator=True)

Table 36.2: SimpleImputer attributes after fitting.

36.3.1 Full Implementation

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer

# Create dataset with missing numerical values
np.random.seed(42)
n = 100
df = pd.DataFrame({
'age': np.random.normal(35, 10, n),
'income': np.random.lognormal(10.5, 0.8, n), # Right-skewed
'score': np.random.normal(70, 15, n)
})

# Introduce MCAR missing values (15%)
for col in df.columns:
mask = np.random.random(n) < 0.15
df.loc[mask, col] = np.nan

print("=== MISSING VALUES BEFORE ===")
print(df.isnull().sum())
print()

# ■■ Mean Imputation ■■
imputer_mean = SimpleImputer(strategy='mean')
df_mean = pd.DataFrame(
imputer_mean.fit_transform(df),
columns=df.columns
)
print("Mean imputation statistics:", imputer_mean.statistics_)

# ■■ Median Imputation ■■
```

```
imputer_median = SimpleImputer(strategy='median')
df_median = pd.DataFrame(
imputer_median.fit_transform(df),
columns=df.columns
)
print("Median imputation statistics:", imputer_median.statistics_)

print("\n=== MISSING VALUES AFTER ===")
print(f"Mean imputer: {df_mean.isnull().sum().sum()} remaining")
print(f"Median imputer: {df_median.isnull().sum().sum()} remaining")

# Compare distributions
print("\n=== DISTRIBUTION COMPARISON ===")
for col in df.columns:
print(f"\n{col}:")
print(f" Original mean: {df[col].mean():.2f}")
print(f" Mean-imputed: {df_mean[col].mean():.2f}")
print(f" Median-imputed: {df_median[col].mean():.2f}")
print(f" Original median: {df[col].median():.2f}")
print(f" Original skew: {df[col].skew():.2f}")
```

Listing 36.1: Mean vs Median imputation with SimpleImputer.

36.4 Column-Wise Imputation

SimpleImputer computes a **separate** statistic for each feature (column). This is the correct behaviour: each feature has its own distribution, so using a single global mean for all features would be wrong. The `statistics_` attribute reveals the per-feature imputation value after fitting.

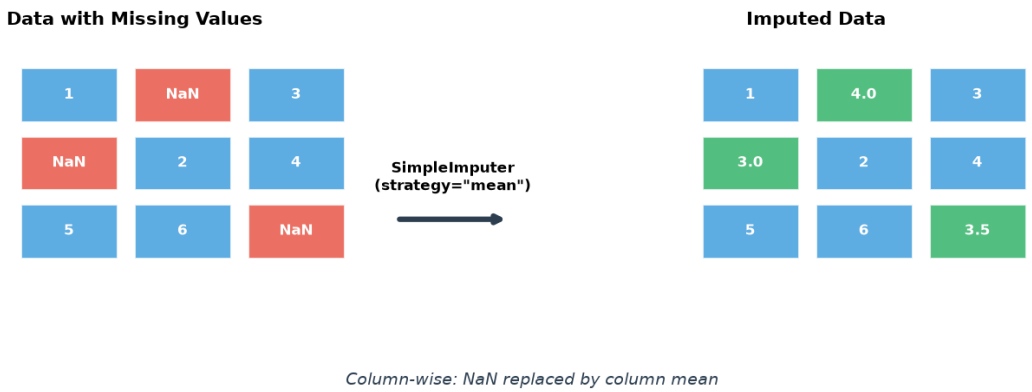


Figure 36.2: SimpleImputer with strategy="mean" replaces each NaN with the mean of its own column. Column 1 mean = 3.0, column 2 mean = 4.0, column 3 mean = 3.5.

36.5 Statistical Implications

Mean Imputation Distorts Variance
Replacing missing values with the column mean **reduces the variance** of the feature. All imputed values are identical (the mean), so they add no spread. This artificial narrowing of the distribution can make relationships appear stronger than they really are and can underestimate standard errors in statistical models.

Median imputation has a similar problem: it reduces variance and can create artificial spikes at the median value. Both methods treat imputed values as if they were observed with certainty, ignoring the uncertainty of the estimate.

Better Alternatives
Methods that preserve the distribution better include: random sample imputation (Chapter 38), KNN imputation (Chapter 39), and iterative/MICE imputation (Chapter 40). These methods produce more realistic imputed values by considering the relationships between features.

36.6 Key Takeaways

★ SimpleImputer replaces missing values with a per-column statistic (mean, median, most_frequent, or constant). Mean works for symmetric data; median is robust to skewness and outliers. Both reduce variance — a limitation to remember when interpreting model results.

36.7 Interview Questions

1. Why is median preferred over mean for skewed distributions?
2. What does SimpleImputer.statistics_ contain after fitting?
3. How does mean imputation affect the variance of a feature?
4. Can SimpleImputer handle categorical data? Which strategy would you use?
5. What is the difference between fit() and transform() in SimpleImputer?

CHAPTER 37

Missing Categorical Data

Most Frequent, Constant, Missing Category Strategies

Learning Objectives

Strategies for categorical missing data; most_frequent vs constant vs missing category; SimpleImputer for strings; consequences of each approach.

37.1 Missing Categorical Values

Categorical features can also contain missing values. Examples include a "city" column where some rows are blank, a "gender" column with unrecorded responses, or a "product_category" column where the category was not assigned. The strategies for handling missing categorical data differ from numerical imputation because we cannot compute a "mean" of text labels.

37.2 Strategies for Missing Categorical Data



Figure 37.1: Four strategies for handling missing categorical data. Each produces a different distribution of categories after imputation.

Strategy	What it does	When to use	Risk
Most Frequent	Replace NaN with the modal category	Missingness is low and one category dominates	Inflates the dominant category
Constant	Replace NaN with a fixed string (e.g., "Unknown")	Missingness might be informative	Adds an artificial category
Missing Category	Create a new "(Missing)" category	The fact that a value is missing may predict the target	High cardinality if applied to many features
Drop	Remove rows with missing categories	Very low missingness and MCAR	Data loss (same as CCA)

Table 37.1: Strategies for missing categorical data.

37.3 Implementation

37.3.1 Most Frequent Imputation

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Categorical data with missing values
df = pd.DataFrame({
'city': ['NYC', 'LA', 'NYC', np.nan, 'Chicago', np.nan,
'NYC', 'Houston', np.nan, 'LA', 'NYC', 'Chicago'],
'gender': ['M', 'F', np.nan, 'F', 'M', 'M',
np.nan, 'F', 'M', np.nan, 'F', 'M'],
'subscription': ['Basic', 'Premium', 'Basic', np.nan, 'Premium',
'Basic', 'Premium', np.nan, 'Basic', 'Premium', 'Basic', 'Premium']
})

print("=== MISSING VALUES ===")
print(df.isnull().sum())
print()

# Most frequent imputation
imputer = SimpleImputer(strategy='most_frequent')
df_imputed = pd.DataFrame(
imputer.fit_transform(df),
columns=df.columns
)

print("Most frequent values (statistics_):", imputer.statistics_)
print("\n=== AFTER MOST FREQUENT IMPUTATION ===")
print(df_imputed)
print("\nCategory counts after imputation:")
for col in df.columns:
print(f"\n{col}:")
print(df_imputed[col].value_counts())
```

Listing 37.1: Most frequent imputation for categorical data.

37.3.2 Constant / Missing Category Imputation

```
# Constant imputation — replace with a designated string
imputer_const = SimpleImputer(strategy='constant', fill_value='Unknown')
df_const = pd.DataFrame(
imputer_const.fit_transform(df),
columns=df.columns
)
print("=== CONSTANT IMPUTATION (fill_value='Unknown') ===")
print(df_const)
print()

# Manual approach: create a missing category
df_missing_cat = df.copy()
for col in df.columns:
df_missing_cat[col] = df_missing_cat[col].fillna('(Missing)')

print("=== MISSING CATEGORY APPROACH ===")
print(df_missing_cat)
print("\nCategory counts with missing category:")
for col in df.columns:
```

```
print(f"\n{col}:")
print(df_missing_cat[col].value_counts())
```

Listing 37.2: Constant and missing-category imputation.

37.4 Consequences of Different Approaches

37.4.1 Most Frequent Imputation

- **Pro:** Simple, preserves the existing category set.
- **Con:** Inflates the frequency of the most common category. If "NYC" was already 40% of the data and you impute 20 more NaNs as "NYC", it might become 55% — distorting the true distribution.
- **Con:** Breaks the relationship between the feature and other features. If missingness depends on another feature (MAR), ignoring that dependence introduces bias.

37.4.2 Constant / Missing Category

- **Pro:** Preserves information that a value was missing (the missingness itself might be predictive).
- **Pro:** Does not inflate any existing category.
- **Con:** Adds a new category, increasing cardinality. For high-cardinality features, this can be problematic for one-hot encoding.
- **Con:** Tree-based models can handle the new category natively, but linear models require encoding — and the encoding of "(Missing)" carries no inherent ordinal meaning.

> **Common Mistake**
> Do not use `strategy="mean"` or `"median"` on categorical (string) columns. These strategies are designed for numerical data. Scikit-learn will either raise an error or produce meaningless results. Always use `"most_frequent"` or `"constant"` for categorical features.

37.5 Key Takeaways

> **★ For missing categorical data, use most_frequent imputation (SimpleImputer strategy="most_frequent") when the dominant category is appropriate, or create a dedicated "(Missing)" category when the missingness itself may be informative. Never apply mean/median strategies to categorical columns.**

CHAPTER 38

Missing Indicator and Random Sample Imputation

Missingness Information, Distribution Preservation

Learning Objectives
Why missingness itself can be informative; MissingIndicator and add_indicator; random sample imputation; distribution preservation; limitations.

38.1 Missing Indicators

Sometimes the **fact that a value is missing** is itself predictive. For example, in a medical dataset, patients who did not report their income might systematically differ from those who did. A **missing indicator** is a binary feature (0 or 1) that records whether the original value was missing. This preserves the missingness information that would otherwise be destroyed by imputation.

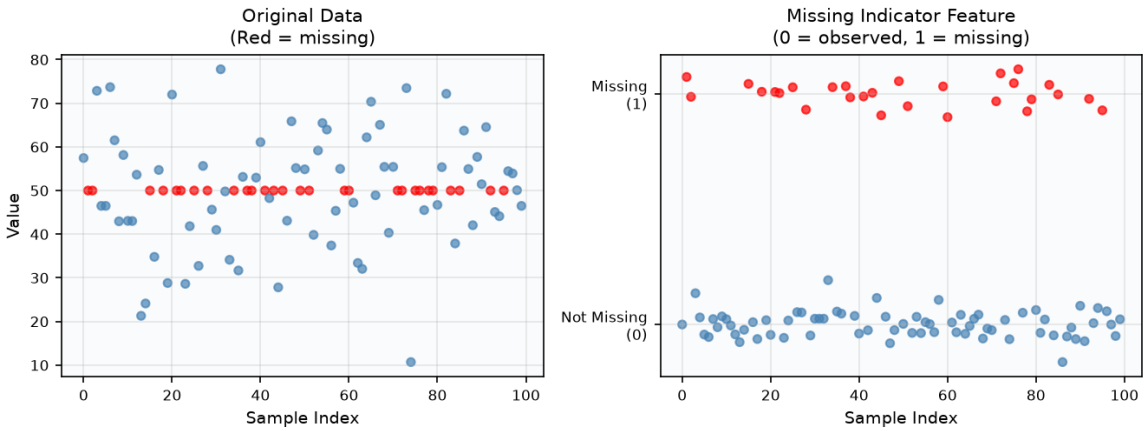


Figure 38.1: Left: original data with missing values (red dots). Right: the corresponding missing indicator feature — 1 where the original was missing, 0 where it was observed.

38.2 MissingIndicator in Scikit-learn

Parameter	Default	Description
features	"missing-only"	Which columns to generate indicators for: "missing-only" (only columns with NaN) or "all"
error_on_new	True	If True, raises error on unseen features at transform time

Table 38.1: MissingIndicator parameters.

```
import numpy as np
import pandas as pd
from sklearn.impute import MissingIndicator, SimpleImputer

df = pd.DataFrame({
'age': [25, 30, np.nan, 45, np.nan, 38, 52, np.nan],
'income': [50000, np.nan, 60000, np.nan, 75000, 80000, np.nan, 90000],
'score': [85, 92, 78, np.nan, 88, 95, 72, np.nan]
})

# Standalone MissingIndicator
indicator = MissingIndicator(features='missing-only')
indicator.fit(df)
missing_features = indicator.transform(df)

print("Missing indicator columns shape:", missing_features.shape)
print("Indicator for first 3 features:")
print(missing_features)

# ■■ add_indicator=True in SimpleImputer ■■
```

```
imputer = SimpleImputer(strategy='mean', add_indicator=True)
df_imputed = imputer.fit_transform(df)

print(f"\nOriginal shape: {df.shape}")
print(f"Imputed shape: {df_imputed.shape}")
print(f"(Added {df_imputed.shape[1] - df.shape[1]} indicator columns)")
print(f"\nImputed data:")
print(pd.DataFrame(df_imputed,
columns=list(df.columns) + ['age_indicator', 'income_indicator', 'score_indicator']))
```

Listing 38.1: MissingIndicator and add_indicator in SimpleImputer.

38.3 Random Sample Imputation

Random sample imputation fills each missing value by randomly selecting an observed value from the same feature. Unlike mean/median imputation, this preserves the original distribution of the feature because it draws from the actual observed values.

The algorithm is straightforward:

1. For each feature, collect all non-missing values.
2. For each missing value, randomly sample one value from the observed set (with replacement).
3. Replace the missing value with the sampled value.

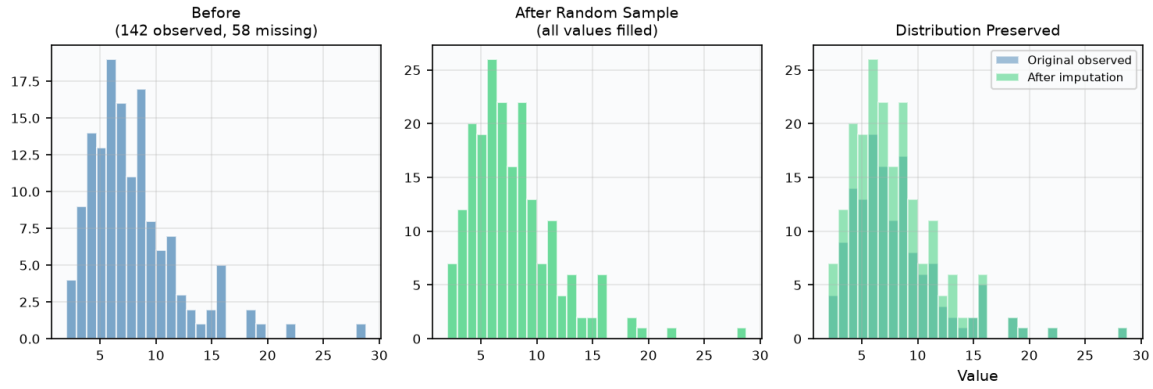


Figure 38.2: Random sample imputation preserves the distribution shape. Left: observed data before imputation. Centre: after random sample imputation. Right: the two distributions overlap closely.

38.3.1 Implementation

```
import numpy as np
import pandas as pd

def random_sample_imputation(df, random_state=42):
"""Fill missing values by sampling from observed values of the same column."""
np.random.seed(random_state)
df_imputed = df.copy()
for col in df.columns:
# Get observed (non-missing) values
observed = df[col].dropna().values
if len(observed) == 0:
continue # Skip all-missing columns
# Find missing positions
missing_mask = df[col].isna()
n_missing = missing_mask.sum()
# Sample with replacement from observed values
sampled = np.random.choice(observed, size=n_missing, replace=True)
# Fill
df_imputed.loc[missing_mask, col] = sampled
return df_imputed

# Example
np.random.seed(42)
df = pd.DataFrame({
'income': np.random.lognormal(10.5, 0.8, 200),
'age': np.random.normal(40, 12, 200),
'score': np.random.normal(70, 15, 200)
})
```

```
# Introduce missing values
for col in df.columns:
mask = np.random.random(200) < 0.2
df.loc[mask, col] = np.nan

print(f"Missing before: {df.isnull().sum().sum()}")

# Impute
df_filled = random_sample_imputation(df)
print(f"Missing after: {df_filled.isnull().sum().sum()}")

# Compare distributions
print("\n=== DISTRIBUTION COMPARISON ===")
print(f"{'Col':<10} {'Obs Mean':>10} {'Filled Mean':>12} {'Obs Std':>10} {'Filled Std':>11}")
for col in df.columns:
obs = df[col].dropna()
print(f"{col:<10} {obs.mean():>10.2f} {df_filled[col].mean():>12.2f} "
f"{obs.std():>10.2f} {df_filled[col].std():>11.2f}")
```

Listing 38.2: Random sample imputation implementation.

38.4 Advantages and Limitations

38.4.1 Missing Indicator

- **Advantage:** Preserves the information that a value was missing, which can be predictive in MAR/MNAR scenarios.
- **Advantage:** Easy to combine with any imputation method via add_indicator=True.
- **Limitation:** Adds extra features, increasing dimensionality. For datasets with many features and low missingness, this can add noise.
- **Limitation:** The indicator is only useful if the model can exploit it. Simple linear models may not benefit as much as tree-based models.

38.4.2 Random Sample Imputation

- **Advantage:** Preserves the original distribution (mean, variance, shape) better than mean/median imputation.
- **Advantage:** Simple to implement and understand.
- **Limitation:** Does not consider relationships between features. Each feature is imputed independently.
- **Limitation:** Introduces randomness — results vary across runs unless you fix the random seed.
- **Limitation:** Can create implausible combinations (e.g., imputing an income that does not match the education level of that observation).

Key Limitation of Random Sample Imputation
Because each feature is imputed independently, random sample imputation can create **internally inconsistent** records. An imputed age of 25 might be paired with an imputed retirement status of "retired" — a combination that would never occur in the real data. Multivariate methods like KNN and MICE (Chapters 39–40) address this by considering feature relationships.

38.5 Key Takeaways

★ Missing indicators preserve the information that a value was missing. Random sample imputation fills NaNs with randomly chosen observed values, preserving the marginal distribution. Both are useful additions to the imputation toolkit but have important limitations regarding feature relationships.

CHAPTER 39

KNN Imputer

K-Nearest Neighbors, Euclidean Distance, nan_euclidean

Learning Objectives
KNN concept; Euclidean distance; nan_euclidean for missing data; KNNImputer parameters; weights; scaling requirement; computational limits.

39.1 Understanding K-Nearest Neighbors

The **K-Nearest Neighbors (KNN)** algorithm is one of the simplest machine-learning algorithms. To predict a value for a new observation, KNN finds the k most similar existing observations (nearest neighbours) and combines their values (e.g., by averaging for regression, or voting for classification).

KNN imputation applies this idea to missing data: instead of using the global mean or median, it uses the values of the k most similar complete observations to estimate the missing value. This means the imputed value respects the relationships between features.

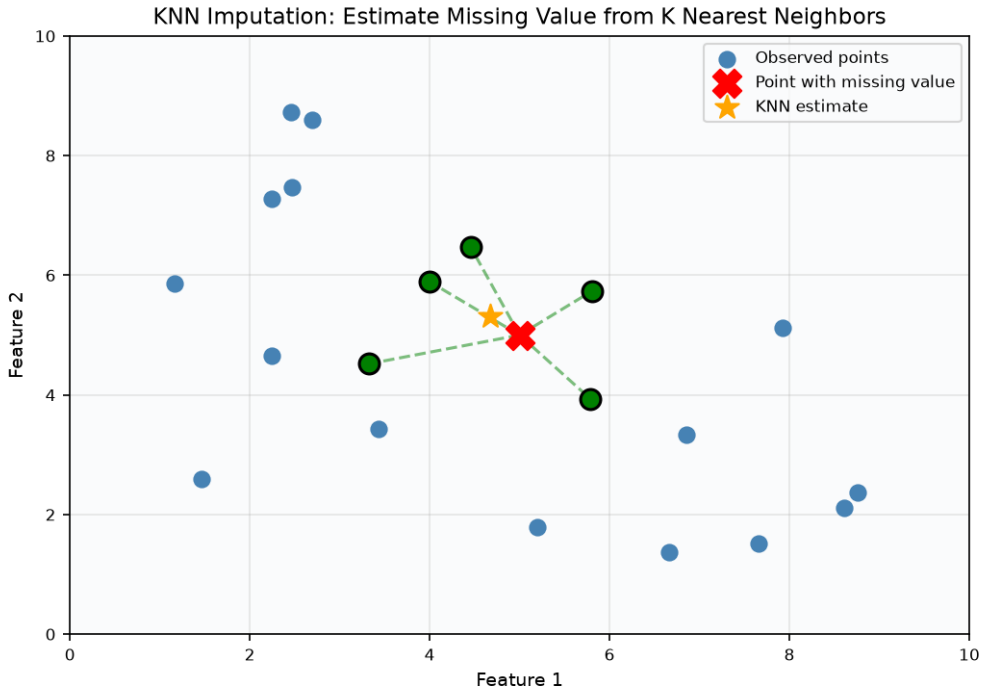


Figure 39.1: KNN imputation concept. The red X marks an observation with a missing value. The 5 nearest neighbours (green circles) are identified, and the missing value is estimated from their values (orange star).

39.2 Mathematical Foundation: Distance

KNN relies on a **distance metric** to determine which observations are "near" each other. The most common metric is **Euclidean distance**:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2} \tag{39.1}$$

where x and y are two observations with p features each. For two-dimensional points (x■, y■) and (x■, y■), this simplifies to:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{39.2}$$

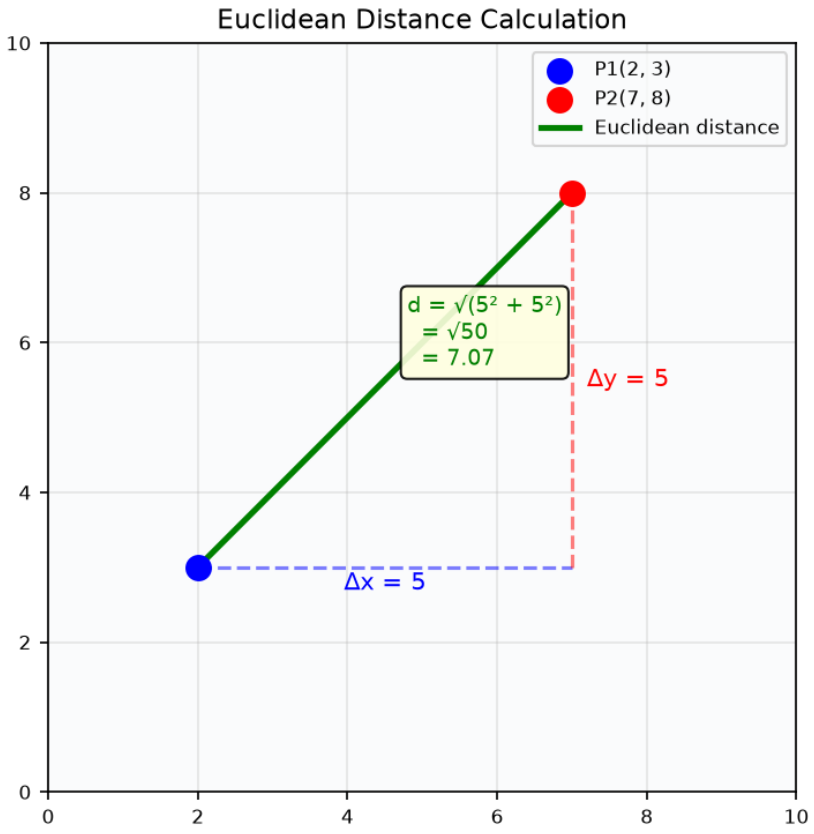


Figure 39.2: Euclidean distance between P1=(2,3) and P2=(7,8). The horizontal and vertical differences form the legs of a right triangle; the distance is the hypotenuse.

39.2.1 Distance with Missing Values

A critical subtlety in KNN imputation: when computing distances between observations, some feature values may be missing. Scikit-learn's KNNImputer handles this by computing the distance using **only the features that are observed in both observations**. Formally, for observations x and y, the distance is computed over the subset of features S where both x■ and y■ are observed:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\frac{p}{|S|}} \cdot \sqrt{\sum_{j \in S}(x_j - y_j)^2} \tag{39.3}$$

The factor p/|S| rescales the distance to account for the missing features, so that distances computed from fewer features are not artificially small.

39.3 KNNImputer in Scikit-learn

Parameter	Default	Description
n_neighbors	5	Number of neighbors to use for imputation
weights	"uniform"	Weight function: "uniform" (equal) or "distance" (inverse distance)
metric	"nan_euclidean"	Distance metric designed for data with NaN values
copy	True	If True, creates a copy; if False, imputes in place

Table 39.1: KNNImputer parameters.

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer

# Create a small example to trace the algorithm manually
data = np.array([
[1.0, 2.0, np.nan],
[2.0, 3.0, 6.0],
[3.0, np.nan, 8.0],
[4.0, 5.0, 10.0],
[5.0, 6.0, np.nan],
[np.nan, 7.0, 12.0]
])

print("Original data:")
print(pd.DataFrame(data, columns=['A', 'B', 'C']))
print()

# Apply KNN imputation with k=2
imputer = KNNImputer(n_neighbors=2, weights='distance')
imputed = imputer.fit_transform(data)

print("After KNN imputation (k=2, weights='distance'):")
print(pd.DataFrame(imputed, columns=['A', 'B', 'C']))
print()

# Trace one imputation manually
# For the missing C in row 0:
# Distance from row 0 to each complete row (using available features)
print("=== MANUAL TRACE: Imputing C[0] (missing) ===")
row0 = data[0] # [1.0, 2.0, NaN]
print(f"Row 0 (with missing): {row0}")

# Compute nan_euclidean distance to each other row
for i in range(1, len(data)):
other = data[i]
# Features observed in both: A and B (C is missing in row 0)
mask = ~np.isnan(row0) & ~np.isnan(other)
n_features_total = data.shape[1]
n_observed = mask.sum()
if n_observed == 0:
print(f" Row {i}: no common features")
continue
sq_diff = np.sum((row0[mask] - other[mask]) ** 2)
distance = np.sqrt(n_features_total / n_observed) * np.sqrt(sq_diff)
print(f" Row {i} {other}: distance = {distance:.4f} (using {n_observed} features)")
```

Listing 39.1: KNNImputer with manual distance trace.

39.4 Weights Parameter

The `weights` parameter controls how neighbouring observations contribute to the imputed value:

Weights	Formula	Behaviour
"uniform"	Each neighbour contributes equally	Imputed value = simple mean of k nearest neighbours
"distance"	Closer neighbours contribute more (inverse distance)	Imputed value = weighted mean, weighted by 1/d

Table 39.2: KNNImputer weights options.

With `weights="distance"`, the imputed value is computed as:

$$\hat{x} = \frac{\sum_{i \in N_k} \frac{x_i}{d_i}}{\sum_{i \in N_k} \frac{1}{d_i}} \tag{39.4}$$

where N■ is the set of k nearest neighbours and d■ is the distance to neighbour i. Closer neighbours (smaller d■) receive higher weight.

39.4.1 Full Pipeline Example

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Realistic dataset
np.random.seed(42)
n = 500
X = pd.DataFrame({
'age': np.random.normal(40, 12, n),
'income': np.random.lognormal(10.5, 0.7, n),
'credit_score': np.random.normal(700, 50, n),
'debt_ratio': np.random.beta(2, 5, n),
'years_employed': np.random.exponential(8, n)
})

# Introduce MAR missingness (age missing more for low income)
for i in range(n):
if X.loc[i, 'income'] < X['income'].quantile(0.3):
if np.random.random() < 0.3:
X.loc[i, 'age'] = np.nan
if np.random.random() < 0.1:
X.loc[i, 'credit_score'] = np.nan
if np.random.random() < 0.15:
X.loc[i, 'debt_ratio'] = np.nan

print("Missing per column:")
print(X.isnull().sum())
print(f"\nTotal missing: {X.isnull().sum().sum()}")

# KNN imputation with different k values
for k in [3, 5, 10]:
imputer = KNNImputer(n_neighbors=k, weights='distance')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)
print(f"\nk={k}: imputed shape={X_imputed.shape}, "
```

```
f"age mean={X_imputed['age'].mean():.2f}, "
f"credit mean={X_imputed['credit_score'].mean():.1f}")
```

Listing 39.2: KNN imputation pipeline with different k values.

39.5 Advantages, Disadvantages and Computational Cost

39.5.1 Advantages

- **Respects feature relationships:** Unlike mean/median imputation, KNN considers all features when estimating a missing value.
- **No distributional assumption:** Does not assume the data follows any specific distribution.
- **Non-parametric:** Makes no assumptions about the functional form of the data-generating process.
- **Intuitive:** The imputed value is based on actual similar observations.

39.5.2 Disadvantages

- **Computationally expensive:** Requires computing distances between all pairs of observations: $O(n^2 \times p)$ time. For large datasets, this is prohibitive.
- **Memory intensive:** Must store the entire dataset for distance computation.
- **Sensitive to feature scaling:** Features with larger ranges dominate the distance calculation. **Always scale features before KNN imputation.**
- **Sensitive to k:** Too small → noisy estimates; too large → oversmoothing. The optimal k depends on the dataset.
- **Curse of dimensionality:** In high-dimensional spaces, all points become roughly equidistant, reducing the discriminative power of neighbours.

> **Always Scale Before KNN**
> KNN uses distance metrics, so features with larger scales dominate. If one feature ranges from 0–1,000,000 (income) and another from 0–1 (debt ratio), the income feature will completely determine the nearest neighbours. Apply StandardScaler or MinMaxScaler **before** KNN imputation.

39.6 Key Takeaways

> **★ KNNImputer estimates missing values from the k most similar observations using nan_euclidean distance. It respects feature relationships but is computationally expensive and sensitive to feature scaling. Always scale features before applying KNN imputation, and choose k through cross-validation.**

39.7 Interview Questions

1. How does KNNImputer handle missing values when computing distances between observations?
2. Why must you scale features before KNN imputation?
3. What is the computational complexity of KNN imputation?
4. Compare uniform vs distance weights in KNNImputer.
5. When would you choose KNN imputation over mean imputation?

CHAPTER 40

MICE and Iterative Imputation

Chained Equations, Multivariate Imputation, Convergence

Learning Objectives
Why univariate imputation is insufficient; MICE algorithm; IterativeImputer; convergence; comparison with KNN; assumptions and limitations.

40.1 Why Univariate Imputation May Be Insufficient

Mean, median, and most-frequent imputation are **univariate** methods: they use only the information from a single feature to estimate missing values. Random sample imputation is also univariate. KNN imputation begins to use multivariate information but is limited to the k nearest neighbours.

In many real-world datasets, features are **correlated**. Income is correlated with education. Age is correlated with years of experience. Blood pressure is correlated with weight and age. When features are correlated, knowing the values of other features gives you information about the missing value. **Multivariate imputation** methods exploit these relationships.

40.2 MICE: Multivariate Imputation by Chained Equations

MICE (also called the chained equations approach or iterative imputation) was proposed by van Buuren, Boshuizen, and Knook (1999). The core idea is elegant: each feature with missing values is treated as the **target** of a regression model, using all other features as **predictors**.

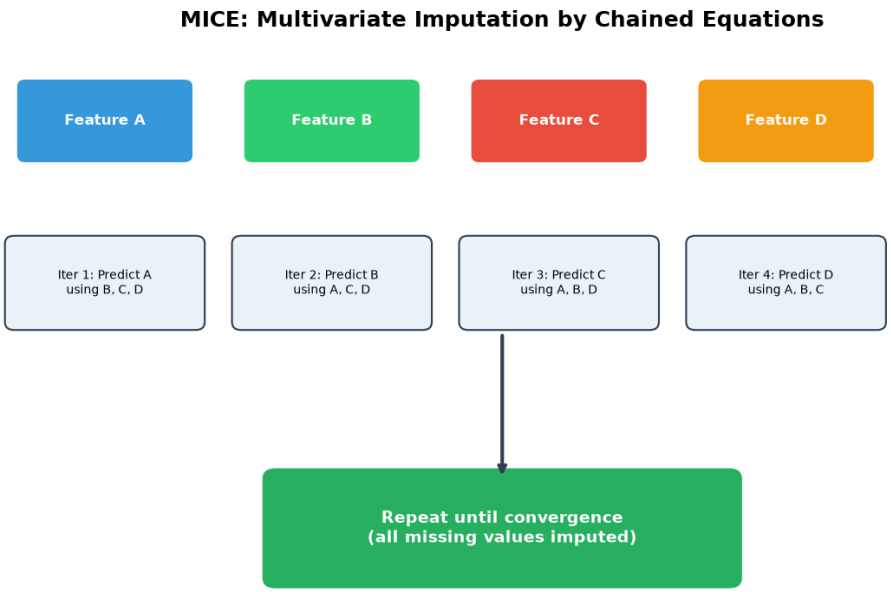


Figure 40.1: The MICE iterative process. Each feature with missing values is predicted using the other features. The process cycles through all features repeatedly until convergence.

40.3 The MICE Algorithm

The algorithm proceeds as follows:

1. **Initial imputation:** Fill all missing values with a simple method (e.g., mean or median) to create a complete initial dataset.
2. **For each feature j with missing values:**
   - Temporarily remove the observed values of feature j (treat them as missing).
   - Build a regression model: feature j is the target; all other features are predictors.
   - Predict the missing values of feature j using the model.
   - Replace the imputed values of feature j with these new predictions.

3. Cycle through all features with missing values. Repeat step 2 for each such feature. One full pass through all features is called one **iteration**.

4. Repeat iterations until the imputed values stabilise (converge). Convergence is typically assessed by monitoring the change in imputed values between iterations.

40.3.1 Mathematical Intuition

Suppose we have three features A, B, C, and feature A has missing values. In the first iteration, we fit a regression model:

$$A_{\text{pred}} = f(B, C) \tag{40.1}$$

and use it to predict the missing values of A. In the next step, if B also has missing values, we fit:

$$B_{\text{pred}} = g(A_{\text{new}}, C) \tag{40.2}$$

using the **updated** values of A (including the newly imputed ones). This sequential updating — where each feature's imputation uses the most recent estimates of other features — is what makes MICE "chained equations".

40.4 IterativeImputer in Scikit-learn

Scikit-learn implements MICE as `IterativeImputer` in `sklearn.impute`. As of Scikit-learn 1.x, it is considered experimental and requires importing from `sklearn.experimental`.

Parameter	Default	Description
estimator	BayesianRidge()	The regression model used to predict each feature
max_iter	10	Maximum number of imputation rounds
random_state	None	Controls randomness for reproducibility
initial_strategy	"mean"	Initial imputation: "mean", "median", "most_frequent"
imputation_order	"ascending"	Order in which features are imputed: "ascending", "descending", "arabic", "random"
sample_posterior	False	If True, uses sampling from posterior; if False, uses point estimates
tol	1e-3	Convergence tolerance: stop when change < tol
n_nearest_features	None	Number of features to use per estimator (for speed)

Table 40.1: IterativeImputer parameters.

```
import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import BayesianRidge

# Dataset with correlated features and missing values
np.random.seed(42)
n = 300

# Create correlated features
education = np.random.normal(14, 3, n) # Years of education
income = 30000 + 5000 * education + np.random.normal(0, 15000, n)
experience = np.random.normal(10, 5, n)
age = 22 + experience + np.random.normal(0, 3, n)
credit_score = 600 + 10 * education + 2 * income/1000 + np.random.normal(0, 40, n)

df = pd.DataFrame({
'education': education,
'income': income,
'experience': experience,
'age': age,
'credit_score': credit_score
})

# Introduce missing values (MAR: less educated more likely to skip income)
for i in range(n):
if education[i] < 12 and np.random.random() < 0.4:
df.loc[i, 'income'] = np.nan
if np.random.random() < 0.1:
df.loc[i, 'credit_score'] = np.nan
if np.random.random() < 0.08:
df.loc[i, 'experience'] = np.nan
```

```
print("Missing per column:")
print(df.isnull().sum())

# Apply MICE imputation
imputer = IterativeImputer(
estimator=BayesianRidge(),
max_iter=10,
random_state=42,
sample_posterior=True
)
df_imputed = pd.DataFrame(
imputer.fit_transform(df),
columns=df.columns
)

print(f"\nShape before: {df.shape}, after: {df_imputed.shape}")
print(f"Missing after: {df_imputed.isnull().sum().sum()}")

# Compare original and imputed correlations
print("\n=== CORRELATION COMPARISON ===")
print("Original (observed pairs):")
print(df.corr().round(3))
print("\nAfter MICE imputation:")
print(df_imputed.corr().round(3))
```

Listing 40.1: MICE imputation with IterativeImputer.

40.5 Comparison: KNN vs MICE

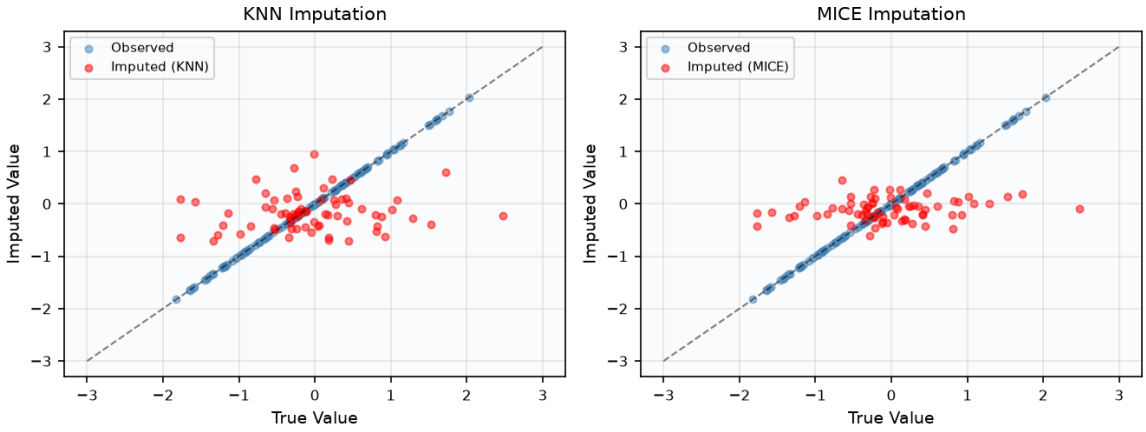


Figure 40.2: KNN vs MICE imputation on correlated data. Both methods recover the relationship between true and imputed values, but MICE tends to produce slightly better-calibrated estimates when features are strongly correlated.

Aspect	KNNImputer	IterativeImputer (MICE)
Approach	Find k similar observations, average their values	Fit regression model per feature, predict missing values
Feature relationships	Implicitly via distance metric	Explicitly via regression coefficients
Computation	$O(n^2 \times p)$ — expensive for large n	$O(\text{iterations} \times n \times p^2)$ — depends on max_iter
Scalability	Poor for large datasets	Better for large datasets
Hyperparameters	n_neighbors, weights	estimator, max_iter, imputation_order
Distribution preservation	Good (samples from observed values)	Good (regression predictions vary)
Best for	Small-to-medium datasets with moderate correlation	Datasets with strong feature correlations

Table 40.2: KNNImputer vs IterativeImputer comparison.

40.6 Convergence and Stopping Criteria

IterativeImputer stops when one of two conditions is met:

1. The maximum number of iterations (`max_iter`) is reached.
2. The change in imputed values between consecutive iterations falls below `tol` (default 1e-3).

In practice, 5–10 iterations are usually sufficient for convergence. If the imputed values are still changing significantly at max_iter, increase the iteration count or check whether the regression model is appropriate for the feature being imputed.

40.7 Advantages and Assumptions

40.7.1 Advantages

• **Uses all features:** Each feature's imputation leverages information from all other features.

• **Preserves correlations:** The iterative process maintains the multivariate relationships in the data.

• **Flexible estimator:** Any regression model can be used (BayesianRidge, DecisionTree, KNeighbors, etc.).

• **Handles different types:** Different estimators can be used for different feature types.

40.7.2 Assumptions

• **MAR assumption:** MICE assumes data is Missing at Random (missingness depends on observed data, not the missing values themselves).

• **Model specification:** The quality of imputation depends on whether the chosen regression model correctly captures the relationship between features.

• **Convergence:** The algorithm assumes the iterative process converges. For highly complex relationships, convergence may be slow or unstable.

When NOT to Use MICE
Avoid MICE when: (1) the dataset is very large and computation time is critical; (2) features are essentially univariate (no correlations to exploit); (3) the missingness is MNAR (MICE assumes MAR); (4) the relationship between features is highly non-linear and the default linear estimator cannot capture it. In such cases, use a tree-based estimator within IterativeImputer or fall back to simpler methods.

40.8 Key Takeaways

★ MICE treats each feature with missing values as a regression target, using all other features as predictors. The process iterates until convergence, preserving multivariate relationships. Scikit-learn implements this as IterativeImputer. It is more powerful than KNN for correlated data but assumes MAR and requires a suitable regression model.

40.9 Interview Questions

1. Explain the MICE algorithm step by step.

2. How does IterativeImputer differ from KNNImputer?
3. What is the role of the estimator parameter in IterativeImputer?
4. What does the max_iter parameter control, and what is a typical value?
5. What assumption does MICE make about the missingness mechanism?
6. How would you handle a mix of numerical and categorical features with MICE?

CONSOLIDATED REVISION

Quick Reference & Cross-Chapter Review

R.1 Comparison of All Missing-Data Methods

Method	Chapter	Type	Preserves Distribution?	Uses Feature Relations?	Best For
Complete Case Analysis	35	Deletion	N/A (removes rows)	No	MCAR + <5% missing
Mean Imputation	36	Univariate	No (reduces variance)	No	Symmetric distributions
Median Imputation	36	Univariate	Partial (robust to skew)	No	Skewed distributions
Most Frequent (categorical)	37	Univariate	No (inflates mode)	No	Categorical, low missingness
Constant / Missing Category	37	Univariate	Adds category	No	Missingness may be predictive
Missing Indicator	38	Feature addition	N/A (adds feature)	No	MAR/MNAR where missingness matters
Random Sample	38	Univariate	Yes (samples observed)	No	Preserving marginal distributions
KNN Imputation	39	Multivariate	Yes (uses neighbours)	Yes (distance)	Small-medium data with correlations
MICE / Iterative	40	Multivariate	Yes (regression-based)	Yes (regression)	Strong feature correlations

Table R.1: Comprehensive comparison of all missing-data handling methods covered in Chapters 35–40.

R.1.1 Decision Framework

Use this decision process when choosing a missing-data strategy:

1. **What percentage of data is missing?** If <5% and MCAR → CCA may be acceptable. Otherwise proceed to step 2.
2. **What is the missingness mechanism?** MCAR → any method works. MAR → imputation (not deletion). MNAR → advanced methods or missing indicators.
3. **Is the feature numerical or categorical?** Numerical → mean/median/KNN/MICE. Categorical → most_frequent/constant/missing category.
4. **Are features correlated?** Strongly correlated → KNN or MICE. Weakly correlated → simple imputation may suffice.
5. **Is the dataset large?** Very large → prefer mean/median (fast) over KNN/MICE (slow). Small → KNN or MICE feasible.
6. **Add missing indicators?** If missingness itself might be predictive, add indicator features alongside any imputation method.

R.2 Key Formulas Reference

R.2.1 Skewness

$$\text{Skewness} = \frac{E[(X - \mu)^3]}{\sigma^3} \tag{R.1}$$

R.2.2 Box-Cox Transformation

$$T(x; \lambda) = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln(x) & \lambda = 0 \end{cases} \tag{R.2}$$

R.2.3 Yeo-Johnson Transformation ($x \geq 0$)

$$T(x; \lambda) = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln(x+1) & \lambda = 0 \end{cases} \tag{R.3}$$

R.2.4 Euclidean Distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2} \tag{R.4}$$

R.2.5 Cyclical Encoding

$$x_{\sin} = \sin\!\left(\frac{2\pi x}{\text{period}}\right), \quad x_{\cos} = \cos\!\left(\frac{2\pi x}{\text{period}}\right) \tag{R.5}$$

R.2.6 KNN Weighted Imputation

$$\hat{x} = \frac{\sum_{i \in N_k} x_i / d_i}{\sum_{i \in N_k} 1 / d_i} \tag{R.6}$$

R.3 Scikit-learn Classes Reference

Class	Module	Key Parameters	Chapter
PowerTransformer	sklearn.preprocessing	method="yeo-johnson"|"box-cox", standardize=True	31
KBinsDiscretizer	sklearn.preprocessing	n_bins, encode, strategy="uniform"|"quantile"|"kmeans"	32
Binarizer	sklearn.preprocessing	threshold	32
dropna()	pandas.DataFrame	how="any"|"all", subset, thresh	35
SimpleImputer	sklearn.impute	strategy="mean"|"median"|"most_frequent"|"constant", add_indicator	36, 37, 38
MissingIndicator	sklearn.impute	features="missing-only"|"all"	38
KNNImputer	sklearn.impute	n_neighbors, weights="uniform"|"distance", metric="nan_euclidean"	39
IterativeImputer	sklearn.experimental.impute	estimator, max_iter, random_state, initial_strategy, imputation_order	40

Table R.2: Scikit-learn classes used across Chapters 31–40.

R.4 Quick Interview Cheat Sheet

Question	Key Answer
Box-Cox vs Yeo-Johnson?	Box-Cox: x>0 only. Yeo-Johnson: all real x.
Mean vs median imputation?	Mean: symmetric data. Median: skewed/outliers.
MCAR vs MAR vs MNAR?	MCAR: random. MAR: depends on observed. MNAR: depends on missing value.
When is CCA valid?	MCAR + small missing % + large enough dataset.
Why scale before KNN?	Distance metrics are dominated by large-scale features.
What does MICE stand for?	Multivariate Imputation by Chained Equations.
MICE assumes what about missingness?	MAR (Missing at Random).
When to use missing indicators?	When the fact that a value is missing may be predictive.
KNN vs MICE?	KNN: distance-based, slower for large n. MICE: regression-based, uses all features.
Power transform for trees?	No benefit — trees are invariant to monotonic transforms.

Table R.3: Quick interview answers for the most common questions.

End of Machine Learning Notes — Videos 31–40
CampusX — 100 Days of Machine Learning